Chapter 21 — Interview Question Bank

This chapter collects the machine-learning questions that come up again and again in ML-engineer and data-scientist interview loops, as gathered in the big public question lists on GitHub (listed at the end). Each question is worded the way interviewers usually say it, and each has a short answer in this book's own words, so you can revise the whole subject in one place and know you have not missed a classic.

How to use this bank

- **Wording.** Questions are phrased as they are typically asked ("What's the difference between bagging and boosting?"), so you recognise them on the day.
- **★ = asked very often.** A star marks the roughly 90 questions that appear across several of the sources listed at the end and are the classics of almost every ML loop. If you only have one evening, do the starred ones.
- **Answers you can say out loud.** Each answer is 2–6 sentences (or a short code snippet plus a sentence or two). That is about 30–60 seconds of speech, which is the right length for a first answer; the interviewer will then ask a follow-up.
- **→ More.** Where this book explains the topic in depth, a link points to the chapter and section (for example *§6.7* means section 6.7). Questions without a link are answered fully here.
- **Terms are defined at first use.** If a word is new, the answer explains it; the glossary in Ch 20 has the rest.
- **How to drill.** Cover the answer, say yours out loud, then check. Mark the ones you stumbled on and repeat only those the next day. For the coding tasks at the end, type the solution from a blank file in under 15 minutes.

Versions assumed: Python 3.14, scikit-learn 1.9, PyTorch 2.14, NumPy 2.5 (the same as the rest of the book).

---

1. ML basics and the workflow

Q1. ★ What is machine learning, and how is it different from traditional programming? In traditional programming a person writes the rules and the computer applies them to data. In machine learning (ML) you give the computer examples (data, often with the correct answers) and an algorithm learns the rules, stored as the numbers inside a *model*. You choose ML when the rules are too many, too fuzzy or too fast-changing to write by hand (spam, image recognition, recommendations) and when you have enough representative data. → *More:* Ch 1 §1.2 — Learning from data vs writing rules

Q2. ★ What's the difference between AI, machine learning and deep learning? Artificial intelligence (AI) is the broad goal of making machines behave intelligently, by any method, including hand-written rules and search. Machine learning is the subset of AI where behaviour is learned from data. Deep learning is the subset of ML that uses neural networks with many layers, which learn their own features from raw inputs such as pixels or text. → *More:* Ch 1 §1.1 — The AI family tree

Q3. ★ Explain supervised, unsupervised, semi-supervised, self-supervised and reinforcement learning. *Supervised*: every training example has a label (the correct answer) and the model learns input → label. *Unsupervised*: no labels; the model finds structure such as clusters or a compact representation. *Semi-supervised*: a few labels plus many unlabelled examples. *Self-supervised*: labels are made from the data itself, such as predicting the next word or a masked patch; this is how large language models are pretrained. *Reinforcement learning*: an agent acts in an environment and learns from rewards rather than labels. → *More:* Ch 1 §1.4 — Types of learning by supervision

Q4. ★ What's the difference between classification and regression? Give an example of each. Classification predicts a category (spam or not spam, which of 10 digits); regression predicts a number (house price, delivery time). Many models do both: a decision tree outputs a class in one case and a

leaf average in the other. Logistic regression, despite its name, is a classification model because it outputs a class probability. → *More:* Ch 1 §1.5 — Classification vs regression

Q5. ⋆ What is the difference between a parameter and a hyperparameter? Parameters are learned from the training data: the weights of a linear model, the split thresholds of a tree. Hyperparameters are chosen before training and control how learning happens: learning rate, tree depth, number of neighbours k, regularisation strength. You tune hyperparameters by comparing scores on validation data, never on the test set. → *More:* Ch 1 §1.6 — Key terms

Q6. Walk me through the end-to-end machine-learning workflow. Frame the problem (what decision does the prediction drive, and which metric shows success); collect and label data; explore it (EDA, exploratory data analysis); clean it and build features; split into train/validation/test; train a simple baseline, then better models; tune on validation; evaluate once on test; deploy; monitor and retrain. Most real time goes into data and evaluation, not model choice. → *More:* Ch 1 §1.9 — The end-to-end ML workflow

Q7. What's the difference between batch (offline) learning and online learning? Batch learning trains on the full dataset at once and is retrained periodically (for example nightly). Online learning updates the model incrementally as each new example or mini-batch arrives, so it adapts quickly to change and handles data that does not fit in memory. Online learning needs guards against bad data quickly corrupting the model. → *More:* Ch 1 §1.7 — Batch vs online learning

Q8. What's the difference between instance-based and model-based learning? Instance-based methods memorise training examples and compare new inputs to them at prediction time (k-nearest neighbours). Model-based methods compress the data into a set of parameters during training and then discard the examples (linear regression, neural networks). Instance-based learning trains instantly but predicts slowly and needs the data stored. → *More:* Ch 1 §1.8 — Instance-based vs model-based learning

Q9. What's the difference between a generative and a discriminative model? A discriminative model learns P(y | x) directly, the boundary between classes (logistic regression, SVMs, most neural classifiers). A generative model learns how the data is produced, P(x | y) and P(y), or P(x) itself, and can create new samples (naive Bayes, Gaussian mixtures, GANs, diffusion models, language models). Discriminative models usually classify better with plenty of data; generative ones cope better with little data and can handle missing inputs. → *More:* Ch 8 §8.3 — Naive Bayes

Q10. ⋆ Why do we split data into training, validation and test sets? The training set fits the parameters. The validation set is used to choose hyperparameters and compare models. The test set is touched once at the end so its score is an unbiased estimate of performance on new data; if you tune on it, it stops being a fair exam. Typical splits are 70/15/15 or 80/10/10, or cross-validation instead of a fixed validation set when data is small. → *More:* Ch 4 §4.7 — Train, validation and test splits

Q11. What are the main challenges (failure modes) in machine learning? Not enough data, unrepresentative data (sampling bias), poor-quality data and labels, irrelevant features, overfitting, underfitting, and data leakage. In production add training/serving skew and drift (the world changing after training). Most of these are data problems, which is why "get better data" usually beats "try a fancier model". → *More:* Ch 1 §1.10 — The main challenges of machine learning

Q12. How do you decide whether a problem needs ML at all? Ask whether simple rules or a lookup already solve it, whether there is a clear pattern to learn, whether enough labelled data exists or can be created, whether mistakes are tolerable and whether the prediction changes a decision. Always build a non-ML baseline (a rule, the majority class, last week's value); if ML cannot beat it by a margin that matters to the business, do not ship ML. → *More:* Ch 15 §15.4 — Scoping a vague AI question

---

2. Bias–variance, overfitting and regularisation

Q13. ⋆ Explain the bias–variance trade-off. Bias is error from wrong assumptions: a model too simple to capture the pattern (a straight line through a curve). Variance is error from sensitivity to the particular training sample: a model so flexible that it fits noise and changes a lot if the data changes. For squared error, expected test error = bias² + variance + irreducible noise. Making a model more complex lowers bias and raises variance, so test error is U-shaped and you pick the complexity at the bottom. → *More:* Ch 6 §6.7 — The bias–variance trade-off

Q14. ⋆ What is overfitting and underfitting? How do you detect them? Overfitting: the model learns noise in the training data, so it scores much better on training than on validation data. Underfitting: the model is too simple and scores poorly on both. Detect them by comparing training and validation scores, and with learning curves (score vs training-set size) or validation curves (score vs a complexity hyperparameter). → *More:* Ch 6 §6.8 — Learning curves and validation curves

Q15. ⋆ How do you prevent overfitting? Get more (or augmented) data; use a simpler model or fewer features; add regularisation (L1/L2 penalties, dropout, weight decay); stop training early when validation loss stops improving; use ensembles such as bagging; and validate with cross-validation so you do not overfit the validation set itself. Also check for leakage: a suspiciously good score is often leakage rather than skill. → *More:* Ch 1 §1.10, Ch 10 §10.7 — Regularization

Q16. How do you fix underfitting? Use a more flexible model (deeper trees, more layers, a kernel), add better features or feature interactions, reduce regularisation, and train longer. If the training score itself is low, more data will not help much; the model needs more capacity or better inputs. → *More:* Ch 6 §6.7

Q17. What is regularisation and why does it work? Regularisation is anything that discourages a model from becoming too complex, usually a penalty added to the loss such as $\lambda \cdot \Sigma w^2$ (L2) or $\lambda \cdot \Sigma |w|$ (L1). The penalty pulls weights toward zero, so the model cannot use huge weights to chase noise. This adds a little bias but removes a lot of variance. The strength λ is a hyperparameter chosen on validation data. → *More:* Ch 5 §5.7 — Regularization: Ridge, Lasso, ElasticNet

Q18. ⋆ What's the difference between L1 and L2 regularisation? Why does L1 give sparse weights? L2 (Ridge) adds the sum of squared weights; it shrinks all weights smoothly and spreads weight among correlated features. L1 (Lasso) adds the sum of absolute weights; it drives some weights exactly to zero, which is built-in feature selection. Geometrically, the L1 constraint region is a diamond whose corners lie on the axes, so the optimum often lands on a corner where some weights are zero; the L2 region is a circle with no corners. From the gradient view, L1's pull toward zero is constant even for tiny weights, while L2's pull shrinks as the weight shrinks. → *More:* Ch 5 §5.7

Q19. What is ElasticNet and when would you use it? ElasticNet combines L1 and L2 penalties with a mixing ratio. Use it when you want sparsity (like Lasso) but have groups of correlated features: plain Lasso tends to pick one of them arbitrarily, while the L2 part lets ElasticNet keep the group together and makes the solution more stable. → *More:* Ch 5 §5.7

Q20. Can you use L2 regularisation for feature selection? Not directly: L2 shrinks weights toward zero but almost never makes them exactly zero, so every feature stays in. You can rank by weight size after scaling, but that is a heuristic. For real selection use L1, tree importances, permutation importance or recursive feature elimination. → *More:* Ch 5 §5.7

Q21. What regularisation techniques exist for neural networks? Weight decay (L2 on the weights), dropout, early stopping, data augmentation, batch or layer normalisation (mild side effect), label smoothing (training on softened targets like 0.9/0.1 instead of 1/0), smaller architectures, and transfer learning from a pretrained model. In practice augmentation and early stopping are the cheapest big wins. → *More:* Ch 10 §10.7 — Regularization: fighting overfitting

Q22. What is early stopping? You track the validation loss during training and stop when it has not improved for a set number of epochs (the *patience*), then restore the best checkpoint. Training longer makes a flexible model fit noise, so the stopping epoch acts like a regularisation hyperparameter that costs nothing to tune. → *More:* Ch 6 §6.10 — Early stopping

Q23. Explain overfitting to a non-technical person. It is like a student who memorises last year's exam answers instead of learning the subject: perfect on questions they have seen, lost on new ones. The model has learned the quirks of its training examples rather than the general pattern, so it looks great in testing on familiar data and disappoints on real, new cases.

Q24. ⋆ What is the curse of dimensionality? As the number of features grows, the space grows exponentially, so data becomes sparse: you need vastly more examples to cover it, and distances between points become nearly equal, which breaks methods that rely on "nearby" points (k-NN, k-means). Models also find more spurious patterns. Remedies: feature selection, dimensionality reduction (PCA, embeddings), regularisation and models that do not depend on raw distances. → *More:* Ch 8 §8.1 — k-NN, Ch 9 §9.5 — PCA

Q25. What is double descent? In very over-parameterised models (huge neural nets), test error can fall, rise near the point where the model can exactly fit the training data, then fall again as the model gets even bigger. It does not contradict bias–variance; it shows that with enough parameters plus implicit regularisation from gradient descent, the model picks smooth solutions among the many that fit. It explains why giant networks generalise despite having more parameters than examples.

---

3. Data, feature engineering, leakage and imbalance

Q26. ⋆ How do you handle missing values? First ask why they are missing: completely at random (MCAR), explainable by other columns (MAR), or related to the missing value itself (MNAR, such as rich people skipping the income question). Then drop rows only if few and random; otherwise impute (median for numbers, a "missing" category for categories), optionally with model-based imputation (KNN, iterative), and add a "was missing" indicator, because missingness itself is often predictive. Fit the imputer on training data only, inside a pipeline. Gradient-boosted trees can handle NaN natively. → *More:* Ch 4 §4.3 — Data cleaning

Q27. How do you detect and handle outliers? Detect with plots (box plots, scatter plots), rules such as values beyond 1.5 × IQR (interquartile range) or $|z| > 3$, or anomaly detectors such as Isolation Forest. Then decide: an outlier may be a data error (fix or drop), a genuine rare case (keep, maybe cap or log-transform), or the very thing you want to find (fraud). Robust choices include median instead of mean, MAE or Huber loss instead of MSE, and tree models. → *More:* Ch 4 §4.3, Ch 9 §9.7 — Anomaly detection

Q28. ⋆ How do you encode categorical variables? One-hot vs label vs target encoding? *One-hot* makes one 0/1 column per category; best for a small number of nominal (unordered) categories. *Label/ordinal* encoding maps categories to integers; fine for ordered categories or for trees, but misleading for linear models because it invents an order. *Target encoding* replaces each category with the average target for that category; great for high-cardinality features (many distinct values, like ZIP codes), but must be computed out-of-fold and smoothed to avoid leakage. *Hashing* maps categories into a fixed number of buckets for unbounded vocabularies; deep models use learned embeddings. → *More:* Ch 4 §4.4 — Encoding categorical variables

Q29. ⋆ What is feature scaling? Normalisation vs standardisation? Which models need it? Standardisation rescales a feature to mean 0 and standard deviation 1; min-max normalisation rescales it into [0, 1]. Models that use distances, gradients or weight penalties need scaling: k-NN, k-means, SVMs, PCA, regularised linear/logistic regression and neural networks. Tree-based models do not, because splits depend only on the order of values. Fit the scaler on training data and apply the same transform to validation, test and production data. → *More:* Ch 4 §4.5 — Feature scaling

Q30. What is feature engineering? Give examples. Creating inputs that make the pattern easier for the model to learn. Examples: ratios (debt / income), date parts (hour of day, day of week, is_holiday), log of skewed amounts, counts and aggregates over time windows ("number of transactions in the last hour"), text features (TF-IDF), interaction features (price × quantity) and distances (to the nearest store). Good features usually come from domain knowledge and often matter more than the model choice. → *More:* Ch 4 §4.6 — Feature creation

Q31. ⋆ What is feature selection and which methods do you know? Choosing a subset of features to reduce overfitting, cost and latency, and to improve interpretability. *Filter* methods score each feature independently (correlation, mutual information, chi-squared). *Wrapper* methods search subsets using a model (forward selection, recursive feature elimination). *Embedded* methods select during training (L1 penalty, tree importances). Do selection inside cross-validation, or the selection step leaks information from the validation folds. → *More:* Ch 4 §4.6, Ch 5 §5.7

Q32. ⋆ What is data leakage? Give examples and say how to prevent it. Leakage is when information that would not be available at prediction time gets into training or evaluation, so offline scores look great and production results disappoint. Examples: a feature recorded after the outcome (a "refund issued" flag for predicting fraud); scaling or feature selection fitted on the full dataset before splitting; random splits of time-series data so the model trains on the future; the same customer or duplicate rows in both train and test; target encoding computed with the row's own label. Prevent it by

splitting first, putting all preprocessing inside a pipeline, using time-based or group-based splits, and building features "as of" the prediction time. → *More:* Ch 4 §4.8 — Data leakage

Q33. Your model has 99% validation accuracy on the first try. What do you check? Suspect leakage or a broken metric before celebrating. Check the class balance (99% may be the majority-class rate), look at the top features for anything that encodes the label or is recorded after it, check for duplicate or same-entity rows across splits, check that the split respects time, and verify the evaluation code by scoring a trivial baseline. Then test on a truly fresh, later slice of data. → *More:* Ch 4 §4.8

Q34. ⋆ How do you handle an imbalanced dataset? First, use the right metric: precision, recall, F1, PR-AUC, not accuracy. Then try, roughly in order: tune the decision threshold; use class weights in the loss; resample the training set (undersample the majority, oversample the minority, or SMOTE, which creates synthetic minority examples by interpolating between neighbours); collect more minority examples; or frame it as anomaly detection when positives are extremely rare. Resample only the training data, and if you change the class ratio, recalibrate the probabilities afterwards. → *More:* Ch 4 §4.9 — Class imbalance

Q35. Why is accuracy a bad metric for imbalanced data? If 99% of transactions are legitimate, a model that always says "legitimate" scores 99% accuracy while catching zero fraud. Accuracy is dominated by the majority class. Use the confusion matrix, precision and recall on the rare class, PR-AUC, or a cost-weighted metric. → *More:* Ch 6 §6.3 — Classification metrics

Q36. Does SMOTE always help? What are its pitfalls? Not always. SMOTE can create unrealistic points in noisy or overlapping regions, does not work naturally for categorical features, and inflates the minority rate so predicted probabilities become too high (they must be recalibrated). It must be applied only inside the training folds; applying it before splitting leaks synthetic copies of validation points into training. With strong models like gradient boosting, class weights plus threshold tuning often do as well. → *More:* Ch 4 §4.9

Q37. How do you split time-series data, and why not randomly? Split by time: train on the past, validate and test on later periods, ideally with a gap between them so features built from recent windows do not overlap. A random split lets the model see the future (and near-duplicate neighbouring days), so validation scores are optimistic. For cross-validation use expanding or rolling windows (scikit-learn's `TimeSeriesSplit`). → *More:* Ch 4 §4.7, Ch 6 §6.6

Q38. What would you do with a high-cardinality categorical feature such as user ID or ZIP code? Options: group rare values into "other"; frequency encoding (replace with how often the value appears); out-of-fold target encoding with smoothing; feature hashing; or a learned embedding in a neural network. Raw IDs often just memorise the training set, so check that the feature still helps on users or items not seen in training. → *More:* Ch 4 §4.4, Ch 13 §13.6 — Embeddings

Q39. Your target (for example price) is heavily right-skewed. What do you do? Prices, incomes and counts are usually right-skewed (a long tail of large values), not normally distributed. Train on log(1 + y) so the model is not dominated by a few huge values and errors become relative rather than absolute, then transform predictions back with exp(·) − 1 (noting this predicts something closer to the median than the mean). Alternatively use a loss suited to the shape, such as Poisson or Tweedie for counts and amounts. → *More:* Ch 4 §4.5

Q40. Why use a `Pipeline` in scikit-learn? A `Pipeline` chains preprocessing steps and the model into one estimator, so `fit` learns scalers, imputers and encoders on training data only and `predict` applies the identical steps to new data. This prevents leakage during cross-validation, keeps training and serving consistent, and lets you tune preprocessing and model hyperparameters together in one grid search. `ColumnTransformer` applies different steps to numeric and categorical columns. → *More:* Ch 4 §4.10 — `Pipeline` + `ColumnTransformer`

Q41. Where do labels come from, and what can go wrong with them? From real outcomes (a chargeback months later), from user behaviour (clicks, which are biased by what was shown and where), from human annotators (costly, inconsistent, so measure agreement), or from weak supervision and LLM labelling (cheap but with correlated errors). Watch for label delay, noisy labels, labels that only exist for items the old system chose to show (selection bias), and label definitions that drift over time. → *More:* Ch 4 §4.1, Ch 16 §16.3

Q42. What is data augmentation? Give examples. Creating extra training examples by applying label-preserving changes to existing ones. Images: flips, crops, rotations, colour jitter, mixup/cutmix.

Text: synonym swaps, back-translation. Audio: noise, time shifts, pitch changes. Tabular: rarely useful beyond SMOTE-style methods. It teaches invariances (a cat is still a cat when flipped) and reduces overfitting; it must only produce changes that really keep the label. → *More:* Ch 11 §11.6 — Data augmentation for images

---

4. Evaluation, metrics, cross-validation and tuning

Q43. ⋆ What is a confusion matrix? A table of predicted class vs actual class. For binary problems it has four cells: true positives (TP, predicted positive and actually positive), false positives (FP, predicted positive but actually negative), false negatives (FN, missed positives) and true negatives (TN). Every classification metric (accuracy, precision, recall, F1, FPR) is a ratio of these four counts. → *More:* Ch 6 §6.3 — Classification metrics

Q44. ⋆ Define precision, recall and F1. When do you prefer precision over recall? Precision = TP / (TP + FP): of the items flagged, how many were right. Recall (sensitivity, true positive rate) = TP / (TP + FN): of the real positives, how many we caught. F1 is their harmonic mean, 2PR / (P + R), high only when both are high. Prefer precision when false alarms are costly (blocking a good customer's card, spam filter hiding real email); prefer recall when misses are costly (cancer screening, fraud you must catch, harmful content). → *More:* Ch 6 §6.3

Q45. ⋆ What is the ROC curve and what does AUC mean? The ROC (receiver operating characteristic) curve plots true positive rate against false positive rate (FP / (FP + TN)) as you sweep the decision threshold from high to low. AUC is the area under it: the probability that a randomly chosen positive is scored higher than a randomly chosen negative. 0.5 is random guessing, 1.0 is perfect ranking. It measures ranking quality across all thresholds and ignores calibration. → *More:* Ch 6 §6.3

Q46. ⋆ ROC-AUC vs PR-AUC: when is PR-AUC better? When positives are rare. FPR divides by the huge number of negatives, so even thousands of false alarms barely move the ROC curve and AUC looks great. Precision divides by the number flagged, so false alarms show up clearly. The baseline for PR-AUC (average precision) is the positive rate (say 0.001), which also makes improvements easier to read. Use ROC-AUC when classes are reasonably balanced or you care about both classes equally. → *More:* Ch 6 §6.3, Ch 16 §16.7

Q47. What are sensitivity, specificity, Type I and Type II errors? Sensitivity = recall = TP / (TP + FN). Specificity = true negative rate = TN / (TN + FP). A Type I error is a false positive (rejecting a true null hypothesis, a false alarm); a Type II error is a false negative (missing a real effect). Lowering the threshold trades Type II errors for Type I errors. → *More:* Ch 2 §2.11, Ch 6 §6.3

Q48. How do you choose the classification threshold? The default 0.5 is rarely right. Choose based on costs and constraints: with calibrated probabilities, flag when p > C_FP / (C_FP + C_FN), where C_FP and C_FN are the costs of a false positive and a false negative. Or pick the threshold that meets a business constraint ("recall ≥ 90%", "at most 500 manual reviews a day") on validation data. Many systems use two thresholds: auto-block above one, send to human review between them. → *More:* Ch 6 §6.3, Ch 6 §6.11

Q49. ⋆ What regression metrics do you know? MSE vs RMSE vs MAE vs R^2? MSE (mean squared error) averages squared errors and punishes large errors heavily; RMSE is its square root, back in the target's units. MAE (mean absolute error) averages absolute errors; it is robust to outliers and its optimal prediction is the median rather than the mean. R^2 is the fraction of variance explained compared with always predicting the mean (1 is perfect, 0 is no better than the mean, negative is worse). MAPE gives percentage error but breaks near zero targets. → *More:* Ch 6 §6.2 — Regression metrics

Q50. What is log loss (cross-entropy) and why use it to evaluate classifiers? Log loss is the average of −log(probability the model gave to the true class). It rewards confident correct predictions and punishes confident wrong ones very heavily (predicting 0.01 for something that happens costs −ln 0.01 ≈ 4.6). Unlike accuracy or AUC it evaluates the probabilities themselves, so it is the right metric when scores feed later calculations, such as bids in an ad auction. → *More:* Ch 2 §2.12, Ch 6 §6.3

Q51. What is model calibration and how do you fix a badly calibrated model? A model is calibrated when its probabilities match reality: of all cases scored 0.7, about 70% are positive. Check with a reliability diagram (predicted vs observed rate per bin) or the Brier score. Fix it on held-out data

with Platt scaling (a logistic regression on the scores) or isotonic regression (a flexible monotonic mapping). Boosted trees, SVMs, naive Bayes and models trained on resampled data are often miscalibrated. → *More:* Ch 6 §6.5 — Calibration

Q52. ⋆ Explain k-fold cross-validation. How do you choose k? Split the data into k equal folds; train on k − 1 folds and validate on the remaining one; repeat k times so each fold is validated once; report the mean and standard deviation of the scores. k = 5 or 10 is the usual compromise: larger k uses more data for training each time (less bias) but costs more and gives more correlated estimates. Leave-one-out (k = n) is for tiny datasets only. → *More:* Ch 6 §6.6 — Cross-validation

Q53. What is stratified, group and time-series cross-validation? *Stratified* k-fold keeps the class proportions the same in every fold, important for imbalanced classes. *Group* k-fold keeps all rows of one entity (a patient, a user) in the same fold, so the model is tested on unseen entities. *Time-series* splitting always validates on data later than the training data. Pick the scheme that mimics how the model will meet new data in production. → *More:* Ch 6 §6.6

Q54. What is nested cross-validation and when do you need it? An inner cross-validation loop tunes hyperparameters, and an outer loop estimates how well the whole "tune then train" procedure generalises. You need it when you want an honest performance estimate from small data without a separate test set; reporting the best score from the same CV you tuned on is optimistically biased. → *More:* Ch 6 §6.6

Q55. ⋆ Grid search vs random search vs Bayesian optimisation? Grid search tries every combination of listed values: fine for one or two hyperparameters, wasteful beyond that. Random search samples combinations at random; for the same budget it tries many more distinct values of each important hyperparameter, so it usually wins when only a few hyperparameters really matter. Bayesian optimisation (Optuna, for example) builds a model of score vs hyperparameters and picks the next trial where improvement looks likely; best when each trial is expensive. Search learning rates and regularisation strengths on a log scale. → *More:* Ch 6 §6.9 — Hyperparameter tuning

Q56. What is a baseline and why should you always build one? A baseline is the simplest reasonable predictor: the majority class, the mean, "same as last week", a hand rule or a plain logistic regression. It tells you whether the problem is learnable, catches bugs (a fancy model that cannot beat the baseline is broken), and gives the business a reference for how much the ML adds. → *More:* Ch 6 §6.1 — Baselines

Q57. ⋆ How do you evaluate a ranking or recommendation model? Explain NDCG, MAP, MRR and precision@k. Precision@k is the fraction of the top k results that are relevant; recall@k is the fraction of all relevant items that appear in the top k. MRR (mean reciprocal rank) averages 1 / (rank of the first relevant result), good when one right answer matters. MAP (mean average precision) averages the precision at each position where a relevant item appears, then averages that over queries. NDCG (normalised discounted cumulative gain) sums graded relevance divided by $\log_2(\text{rank} + 1)$ and normalises by the best possible ordering, so it rewards putting the most relevant items at the top. → *More:* Ch 6 §6.4 — Ranking metrics, Ch 13 §13.11

Q58. Offline metrics improved but the A/B test showed no gain. Why might that be? Common causes: the offline metric is not aligned with the business metric (better log loss on clicks, no effect on revenue); training/serving skew (features differ online); the evaluation data was biased by the old model's choices; leakage inflated the offline score; novelty or position effects; the improvement is real but too small to detect with the traffic you had; or the change affects a segment too small to move the total. Check online feature parity and slice the experiment results. → *More:* Ch 16 §16.7, Ch 16 §16.9

Q59. How do you compare two models properly? Is a 0.5% gain real? Use the same splits for both, look at the spread across folds or bootstrap resamples, and use a paired test (for example a paired t-test on per-fold scores or McNemar's test on the same test cases). A gain smaller than the fold-to-fold noise is not convincing. Also weigh cost: a 0.5% gain may not be worth twice the latency. → *More:* Ch 6 §6.6, Ch 2 §2.11

Q60. What are learning curves and what do they tell you? They plot training and validation scores against the number of training examples. A large gap that narrows as data grows means high variance: more data will help. Both curves flat and low means high bias: more data will not help, you need a better model or features. → *More:* Ch 6 §6.8

Q61. Micro vs macro vs weighted averaging for multiclass metrics? Macro averaging computes the metric per class and takes the plain mean, so small classes count equally. Weighted averaging weights each class by its frequency. Micro averaging pools all TP/FP/FN counts first; for single-label multiclass problems micro-F1 equals accuracy. Use macro when rare classes matter. → *More:* Ch 6 §6.3

Q62. How do you translate an ML metric into a business decision? Attach costs and values to each cell of the confusion matrix (the value of catching a fraud, the cost of a manual review, the cost of an annoyed customer), pick the threshold that maximises expected profit on validation data, and report the result in business units ("saves $X per month at Y reviews per day"). Then confirm with an online experiment. → *More:* Ch 6 §6.11 — From ML metric to business decision

---

5. Linear and logistic regression

Q63. ⋆ Explain linear regression. What loss does it minimise? Linear regression predicts a number as a weighted sum of the features plus an intercept: $\hat{y} = w \cdot x + b$. It is fitted by minimising the mean squared error between predictions and targets (ordinary least squares). Each weight is the expected change in the prediction for a one-unit increase in that feature, holding the others fixed. → *More:* Ch 5 §5.2 — Linear regression: model and loss

Q64. ⋆ What are the assumptions of linear regression? Linearity (the target is a linear function of the features); independent errors; homoscedasticity (constant error variance across predictions); normally distributed errors (needed for confidence intervals and p-values, not for prediction); and no strong multicollinearity. Check them with residual plots (residuals vs fitted should look like random noise) and Q-Q plots. → *More:* Ch 5 §5.5 — Assumptions of linear regression

Q65. ⋆ Normal equation vs gradient descent: when would you use each? The normal equation $\theta = (X^T X)^{-1} X^T y$ gives the exact solution in one step with no learning rate, but costs about $O(n \cdot d^2 + d^3)$ for n rows and d features (forming $X^T X$, then solving it) and needs the data in memory. Gradient descent iterates small steps downhill; it scales to millions of rows and features, works for any differentiable loss, and supports streaming mini-batches, but needs a learning rate and scaled features. Use the closed form for small d, gradient descent otherwise. (Libraries actually solve least squares with QR or SVD, which is more stable than inverting $X^T X$.) → *More:* Ch 5 §5.3 — The normal equation vs gradient descent

Q66. What is multicollinearity and why does it matter? Multicollinearity means features are strongly correlated with each other. Predictions can still be fine, but individual coefficients become unstable (large, sign-flipping, with huge standard errors), so you cannot interpret them. Detect it with the correlation matrix or VIF (variance inflation factor; above 5–10 is a warning). Fix it by dropping or combining features, PCA, or Ridge regularisation. → *More:* Ch 5 §5.5

Q67. Your data has columns x, y and z = x + y. What happens to linear regression? The columns are perfectly collinear, so $X^T X$ is singular (not invertible) and there are infinitely many weight combinations giving the same predictions. The normal equation fails; a library using a pseudo-inverse or gradient descent returns one of the solutions, with meaningless individual weights. If z = x + y + small noise, the problem is solvable but the weights are unstable. Ridge regularisation or dropping z fixes both. → *More:* Ch 5 §5.5

Q68. ⋆ Explain logistic regression. Why is it called "regression" if it classifies? It computes a linear score $z = w \cdot x + b$ and passes it through the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ to get a probability between 0 and 1. It is trained by minimising log loss (equivalently, maximising the likelihood of the labels). The name comes from regressing the log-odds, $\log(p / (1 - p))$, linearly on the features; you get a class by thresholding the probability. → *More:* Ch 5 §5.8 — Logistic regression

Q69. Is logistic regression a linear model? Yes, as a classifier. The probability is a non-linear (sigmoid) function of the features, but the decision boundary $p = 0.5$ is where $w \cdot x + b = 0$, which is a straight line (a hyperplane in many dimensions). It can model curved boundaries only if you add non-linear features such as polynomial terms. → *More:* Ch 5 §5.8

Q70. ⋆ Why does logistic regression use log loss instead of mean squared error? With the sigmoid, MSE gives a non-convex loss surface (multiple flat regions and local minima) and very small gradients when the model is confidently wrong. Log loss is the maximum-likelihood loss for a Bernoulli outcome, is

convex for logistic regression so gradient descent finds the global minimum, and gives the clean gradient $(p - y)\cdot x$. → *More:* Ch 5 §5.8, Ch 2 §2.12

Q71. ⋆ How do you interpret the coefficients of a logistic regression? A coefficient w is the change in log-odds for a one-unit increase in that feature, holding others fixed; e^w is the odds ratio. For example $w = 0.7$ means the odds multiply by $e^{0.7} \approx 2$. Compare magnitudes only after standardising features, and be careful with correlated features, whose coefficients share credit unpredictably. → *More:* Ch 5 §5.10 — Interpreting coefficients

Q72. If one weight is larger than another, is that feature more important? Not necessarily. Weights depend on the feature's units (income in dollars vs thousands), so compare only standardised weights. Even then, correlated features split or swap weight, and a big weight on a rarely non-zero feature may matter little overall. Use permutation importance or SHAP on held-out data for importance. → *More:* Ch 5 §5.10, Ch 19 §19.5

Q73. How does logistic regression handle multiple classes? What is softmax? Either one-vs-rest (train one binary classifier per class, pick the highest score) or multinomial (softmax) regression. Softmax turns a vector of K scores into probabilities that sum to 1: $p_i = e^{z_i} / \Sigma_j e^{z_j}$. It is trained with categorical cross-entropy and is also the standard output layer of neural classifiers. → *More:* Ch 5 §5.9 — One-vs-rest and softmax

Q74. What does the intercept (bias term) mean in a linear model? It is the prediction when every feature is zero, or, if features are standardised, the prediction for an average example. In logistic regression it sets the base log-odds, so it absorbs the class prior. Without it the fitted line is forced through the origin, which is almost never correct. → *More:* Ch 5 §5.10

Q75. What is polynomial regression, and is it still linear regression? You add powers and products of features (x^2, $x\cdot z$) and then fit ordinary linear regression on the expanded features. It is still linear in the parameters, so the same solver works, but it fits curves in the original inputs. High degrees overfit quickly and blow up the number of features, so combine with regularisation. → *More:* Ch 5 §5.6 — Polynomial regression

Q76. What's the difference between linear regression with MSE and with MAE or Huber loss? MSE fits the conditional mean and is very sensitive to outliers because errors are squared. MAE fits the conditional median and ignores how far away outliers are. Huber loss is quadratic for small errors and linear for large ones, a compromise. Quantile loss fits any chosen percentile, useful for prediction intervals such as "90th-percentile delivery time". → *More:* Ch 6 §6.2, Ch 20 §20.6 — Loss functions

---

6. Decision trees, random forests and boosting

Q77. ⋆ How does a decision tree decide where to split? At each node it tries every feature and candidate threshold and keeps the split that most reduces impurity: Gini impurity or entropy for classification, variance (MSE) for regression. It repeats greedily on each child until a stopping rule (maximum depth, minimum samples per leaf). Greedy means it picks the best split now without looking ahead, so the tree is good but not globally optimal. → *More:* Ch 7 §7.1 — Decision trees

Q78. ⋆ Gini impurity vs entropy (information gain)? Gini $= 1 - \Sigma\, p_i^2$, the chance of misclassifying a random sample labelled by the node's class mix. Entropy $= -\Sigma\, p_i \log_2 p_i$; information gain is the parent's entropy minus the weighted entropy of the children. They choose the same split most of the time; Gini is slightly cheaper (no logarithm) and is scikit-learn's default. → *More:* Ch 7 §7.1

Q79. ⋆ Why do decision trees overfit, and how do you stop it? A tree can keep splitting until every leaf is pure, memorising the training data, including noise; a small change in data can produce a very different tree (high variance). Control it with max depth, min samples per leaf or split, max leaf nodes, cost-complexity pruning (`ccp_alpha`), or by averaging many trees in an ensemble. → *More:* Ch 7 §7.2 — Bias–variance of trees

Q80. What are the advantages and disadvantages of a single decision tree? Pros: easy to explain and draw, no scaling needed, handles mixed feature types and non-linear interactions, fast predictions. Cons: unstable and prone to overfitting, axis-aligned staircase boundaries, cannot extrapolate beyond the range of the training targets, and usually less accurate than ensembles. → *More:* Ch 7 §7.1

Q81. ⋆ What is bagging? How does a random forest work? Bagging (bootstrap aggregating) trains many models on bootstrap samples (random samples drawn with replacement) and averages their predictions, which cuts variance. A random forest bags deep decision trees and adds one more trick: at each split it considers only a random subset of features (`max_features`). This decorrelates the trees, so averaging removes more variance than bagging alone. → *More:* Ch 7 §7.3 — Bagging and Random Forests

Q82. Why does a random forest need feature randomness, not just bootstrap samples? If one feature is very strong, every bagged tree splits on it first and the trees become highly correlated; averaging correlated models barely reduces variance. Restricting each split to a random subset of features forces trees to use other features, making their errors less correlated, so the average is much better. → *More:* Ch 7 §7.3

Q83. What is the out-of-bag (OOB) error? Each bootstrap sample leaves out about 37% of the rows (the chance a row is never picked is $(1 - 1/n)^n \approx 1/e$). Each row can therefore be predicted by the trees that did not see it. The average of those predictions gives a free validation estimate without a separate hold-out set. → *More:* Ch 7 §7.3

Q84. Can a random forest overfit by adding more trees? How many trees do you need? Adding trees does not increase overfitting; the error levels off as the average stabilises, and extra trees only cost time and memory. Pick the point where the OOB or validation score stops improving (often a few hundred). A forest can still overfit through its individual trees (very deep trees on noisy data), which you control with depth and leaf-size limits. → *More:* Ch 7 §7.3

Q85. ⋆ What is boosting? Explain gradient boosting. Boosting builds models sequentially, each one focusing on the mistakes of the ensemble so far, and adds them up. Gradient boosting fits each new shallow tree to the negative gradient of the loss with respect to the current predictions (for squared error, simply the residuals), then adds it scaled by a learning rate (shrinkage). Many small corrective steps build a strong model; it mainly reduces bias. → *More:* Ch 7 §7.4 — Boosting

Q86. ⋆ What's the difference between bagging and boosting? Random forest vs gradient boosting? Bagging trains independent models in parallel on resampled data and averages them: it reduces variance, uses deep trees, and is robust with little tuning. Boosting trains models sequentially on the previous errors: it reduces bias, uses shallow trees, and is usually more accurate on tabular data but more sensitive to hyperparameters and noisy labels, and needs early stopping. Random forest is the "safe default"; gradient boosting (XGBoost, LightGBM, CatBoost) is the "win the benchmark" choice. → *More:* Ch 7 §7.2–7.4

Q87. How does AdaBoost differ from gradient boosting? AdaBoost reweights the training examples: after each weak learner (often a one-split "stump"), misclassified examples get higher weight so the next learner focuses on them, and each learner gets a vote based on its accuracy. Gradient boosting instead fits each new learner to the negative gradient of any differentiable loss. AdaBoost turns out to be the same as forward stagewise additive modelling with the exponential loss, so it can be seen as one member of the gradient-boosting family. → *More:* Ch 7 §7.4

Q88. ⋆ What makes XGBoost and LightGBM fast and accurate? XGBoost uses a second-order (gradient and Hessian) approximation of the loss, explicit regularisation on leaf weights (L1/L2) and on the number of leaves (γ), sparsity-aware handling of missing values (each split learns a default direction), and parallel split finding; since version 2.0 its default `tree_method` is `hist`, the same histogram binning LightGBM popularised. By default XGBoost grows trees depth-wise (level by level). LightGBM grows trees leaf-wise (always splitting the leaf with the largest gain), which reaches a lower loss with the same number of leaves but can overfit small data, so you cap `num_leaves` and `min_data_in_leaf`; it also adds GOSS (gradient-based one-side sampling: keep rows with large gradients, sample the rest) and exclusive feature bundling for sparse features. CatBoost adds ordered target statistics for categorical features and symmetric (oblivious) trees. All three support early stopping and native categorical features. → *More:* Ch 7 §7.4

Q89. What are the most important hyperparameters of gradient boosting, and how do you tune them? Number of trees and learning rate (they trade off: smaller learning rate needs more trees; set a small rate such as 0.05 and choose the number of trees by early stopping); tree depth or number of leaves (interaction complexity); minimum samples or child weight per leaf; row and column subsampling; L1/L2 penalties on leaves. Tune depth and leaves first, then subsampling and regularisation, with random or Bayesian search. → *More:* Ch 7 §7.7 — Hyperparameter cheat sheet

Q90. Can you parallelise random forest training? Gradient boosting? Random forest is trivially parallel: each tree is independent. Gradient boosting is sequential across trees because each tree depends on the previous ones, but the work inside one tree (finding the best split across features or histogram bins) is parallelised, which is what XGBoost and LightGBM do. → *More:* Ch 7 §7.3–7.4

Q91. ⋆ How do you get feature importance from tree models, and why can it mislead? Impurity-based importance sums how much each feature reduced impurity across all splits. It is computed on training data, favours high-cardinality and continuous features (they offer more split points), and splits credit unpredictably among correlated features. Better: permutation importance on held-out data (shuffle one feature and measure the score drop) or SHAP values, which attribute each prediction fairly among features. → *More:* Ch 7 §7.3, Ch 19 §19.5 — Explainability

Q92. What are stacking and voting ensembles? Voting combines several different models by majority vote (hard) or averaged probabilities (soft). Stacking trains a meta-model (often a logistic regression) on the out-of-fold predictions of base models, so it learns how much to trust each one. Both help most when the base models make different kinds of errors. → *More:* Ch 7 §7.5 — Voting and stacking

Q93. Why do ensembles usually beat single models? Averaging several models whose errors are not perfectly correlated cancels part of the error: the variance of an average of n independent models falls by a factor of n, and even partly correlated ones help. Boosting additionally reduces bias. The gain depends on diversity, which is why random forests inject randomness and why stacking mixes different model families. → *More:* Ch 7 §7.2–7.5

Q94. When would you choose gradient-boosted trees over a neural network? For tabular data with mixed numeric and categorical columns, up to millions of rows, where features are already meaningful: boosted trees are usually more accurate, need less tuning, train on a CPU, handle missing values, and are easier to explain. Neural networks win on raw unstructured data (images, audio, text) and very large datasets where learned representations pay off. → *More:* Ch 7 §7.6 — When trees beat neural networks

Q95. Why can't tree models extrapolate, and why does that matter for time series? A regression tree predicts a leaf average of training targets, so it can never output a value outside the range it saw. If a trend pushes sales higher than ever before, a tree-based forecaster flattens out at the historical maximum. Remedies: model differences or ratios instead of levels, detrend first, or use a linear component. → *More:* Ch 7 §7.1, Ch 12 §12.10

---

7. k-NN, SVM and naive Bayes

Q96. ⋆ How does k-nearest neighbours work? How do you choose k? To predict for a new point, find the k closest training points (by Euclidean or another distance on scaled features) and take a majority vote (classification) or average (regression), optionally weighted by distance. Small k gives a jagged, high-variance boundary; large k smooths toward the majority (high bias). Choose k by cross-validation; an odd k avoids ties in binary problems. → *More:* Ch 8 §8.1 — k-Nearest Neighbours

Q97. What are the drawbacks of k-NN? Prediction is slow (O(n·d) per query without an index) and needs the whole training set in memory; it is sensitive to feature scaling and irrelevant features; and it degrades in high dimensions where distances become similar. Approximate nearest-neighbour indexes (HNSW, IVF) make it practical at scale, which is how embedding search works. → *More:* Ch 8 §8.1, Ch 13 §13.9

Q98. ⋆ Explain support vector machines. An SVM finds the hyperplane that separates two classes with the largest margin, the gap to the nearest points, which are called support vectors. A wider margin generalises better. The soft-margin version allows some points inside the margin or misclassified, with the hyperparameter C setting the penalty: large C fits training data tightly (low bias, high variance), small C prefers a wider margin. The loss is the hinge loss, max(0, 1 − y·f(x)). → *More:* Ch 8 §8.2 — Support Vector Machines

Q99. ⋆ What is the kernel trick? Many problems are not linearly separable in the original space but are after mapping to a higher-dimensional feature space φ(x). The SVM's dual formulation only needs dot products between examples, so you can replace x·z with a kernel K(x, z) = φ(x)·φ(z) computed directly, without ever building φ. Common kernels: linear, polynomial and RBF (Gaussian), K = exp(−γ‖x − z‖²). → *More:* Ch 8 §8.2

Q100. What do C and gamma control in an RBF SVM? C is the penalty for margin violations (high C: fewer training errors, more complex boundary). Gamma sets how far one training point's influence reaches: high gamma means very local influence and wiggly boundaries that can overfit; low gamma means smooth boundaries. They interact, so tune them together on a log-scale grid, with features standardised. → *More:* Ch 8 §8.2

Q101. When would you use an SVM vs logistic regression? Both are linear classifiers in their basic form; logistic regression gives calibrated-ish probabilities and scales to huge data, and the hinge loss of SVMs focuses only on boundary points. Kernel SVMs are strong on small-to-medium, high-dimensional data (text, bioinformatics) where a non-linear boundary helps, but training cost grows roughly quadratically or worse with the number of samples, so they are rarely used above a few hundred thousand rows. → *More:* Ch 8 §8.2, Ch 8 §8.4

Q102. ⋆ Explain naive Bayes. Why is it "naive", and why does it still work? It applies Bayes' theorem, $P(\text{class} \mid \text{features}) \propto P(\text{class}) \cdot \prod P(\text{feature}_i \mid \text{class})$, assuming features are independent given the class. That assumption is almost never true (words in an email are correlated), which is the "naive" part. It still classifies well because you only need the highest-scoring class to be right, not the exact probabilities; it is fast, needs little data and is a classic text baseline. Its probabilities, however, are usually over-confident. → *More:* Ch 8 §8.3 — Naive Bayes

Q103. What is Laplace (additive) smoothing in naive Bayes? If a word never appeared with a class in training, its estimated probability is zero, and multiplying by zero wipes out all other evidence. Laplace smoothing adds a small count α (often 1) to every word count so no probability is exactly zero. In practice you also sum log-probabilities rather than multiplying probabilities, to avoid numerical underflow. → *More:* Ch 8 §8.3

Q104. How do you choose among classic supervised algorithms? Consider data size and type, the need for interpretability, latency and the shape of the boundary. Start with a fast baseline (logistic or linear regression); use gradient-boosted trees for tabular data; naive Bayes or linear models on TF-IDF for quick text classifiers; k-NN when similarity is the whole point; kernel SVMs on small high-dimensional data. Always compare with cross-validation rather than reputation. → *More:* Ch 8 §8.4 — Choosing a classic supervised algorithm, Ch 20 §20.16

---

8. Unsupervised learning: clustering and dimensionality reduction

Q105. ⋆ How does k-means work? Choose k initial centroids; assign every point to its nearest centroid; move each centroid to the mean of its assigned points; repeat until assignments stop changing. This minimises within-cluster sum of squares (inertia). It always converges, but only to a local optimum, so use k-means++ initialisation (spread-out starting centroids) and several restarts, and scale features first. → *More:* Ch 9 §9.1 — k-means clustering

Q106. ⋆ How do you choose k in k-means? Elbow method: plot inertia against k and look for the bend where adding clusters stops helping much. Silhouette score: for each point, compares the average distance to its own cluster with the distance to the nearest other cluster (from −1 to 1, higher is better). Also gap statistic, stability across resamples, and above all whether the clusters are useful for the business purpose. → *More:* Ch 9 §9.1, Ch 9 §9.9

Q107. What are the limitations of k-means? It assumes roughly spherical clusters of similar size and density, needs k chosen in advance, is sensitive to initialisation, outliers and feature scale, and works only with means, so not with categorical data (k-modes or k-prototypes handle that). For elongated or nested shapes use DBSCAN, Gaussian mixtures or hierarchical clustering. → *More:* Ch 9 §9.1

Q108. How does DBSCAN work, and when would you choose it over k-means? DBSCAN (density-based spatial clustering of applications with noise) marks a point as a core point if at least `min_samples` points lie within radius ε; clusters grow from connected core points, and points reachable from none are labelled noise. Choose it when clusters have arbitrary shapes, when the number of clusters is unknown, or when you want outliers flagged. It struggles when clusters have very different densities and in high dimensions. → *More:* Ch 9 §9.3 — DBSCAN

Q109. What is hierarchical clustering? Agglomerative clustering starts with every point as its own cluster and repeatedly merges the two closest clusters, recording the merges in a tree (dendrogram).

Cutting the tree at a height gives any number of clusters. The linkage rule defines "closest": single (nearest points, makes chains), complete (farthest points), average, or Ward (smallest increase in variance). It costs at least $O(n^2)$ memory, so it suits small datasets. → *More:* Ch 9 §9.2 — Hierarchical clustering

Q110. What is a Gaussian mixture model and how does EM fit it? A GMM assumes the data comes from k Gaussian distributions, each with its own mean, covariance and weight, giving soft (probabilistic) cluster membership and elliptical clusters. Expectation–maximisation alternates: the E-step computes each point's probability of belonging to each component; the M-step re-estimates means, covariances and weights from those soft assignments. k-means is the limit of a GMM with equal spherical covariances and hard assignments. → *More:* Ch 9 §9.4 — Gaussian mixture models and EM

Q111. ⋆ Explain PCA. How do you choose the number of components? Principal component analysis finds orthogonal directions (principal components) along which standardised data varies the most; they are the eigenvectors of the covariance matrix, or equivalently come from the SVD of the centred data. Projecting onto the first few components keeps most of the variance in fewer dimensions. Choose the number by cumulative explained variance (for example 95%), by a scree-plot elbow, or by downstream model performance. → *More:* Ch 9 §9.5 — PCA, Ch 2 §2.3

Q112. Why must you standardise data before PCA? PCA picks directions of maximum variance, and variance depends on units: income in dollars (values in the tens of thousands) would dominate age in years just because its numbers are bigger, and the same distance written in metres instead of kilometres would get a million times more variance. Standardising puts every feature on equal footing. Also centre the data (subtract the mean), which PCA requires. → *More:* Ch 9 §9.5

Q113. What are the limitations of PCA? It is linear, so it misses curved structure; it is unsupervised, so the directions of maximum variance may not be the ones that predict the target; components are mixtures of features and hard to interpret; and it is sensitive to scaling and outliers. For visualisation of non-linear structure use t-SNE or UMAP; for supervised reduction consider LDA. → *More:* Ch 9 §9.5–9.6

Q114. PCA vs LDA vs t-SNE vs UMAP? PCA: unsupervised, linear, keeps global variance, fast, used for compression and pre-processing. LDA (linear discriminant analysis): supervised, linear, finds directions that best separate known classes (at most classes − 1 dimensions). t-SNE: non-linear, preserves local neighbourhoods, for 2-D visualisation only; distances between clusters and cluster sizes in the plot are not meaningful. UMAP: similar purpose to t-SNE, faster, keeps somewhat more global structure and can transform new points. → *More:* Ch 9 §9.6 — t-SNE and UMAP

Q115. What is SVD and where is it used in ML? Singular value decomposition factors any matrix as $A = U\Sigma V^T$: orthogonal directions in the input and output spaces scaled by singular values. Uses: computing PCA, low-rank approximation and compression (keep the top singular values), latent semantic analysis of text, and matrix-factorisation recommenders, where a user–item matrix is approximated by low-rank user and item factors. → *More:* Ch 2 §2.3, Ch 13 §13.5

Q116. How would you detect anomalies without labels? Statistical rules (z-scores, IQR) for single features; Isolation Forest (anomalies are isolated by few random splits); local outlier factor (density lower than neighbours'); one-class SVM; autoencoder reconstruction error (normal data reconstructs well, anomalies do not); or distance to the nearest cluster centre. Validate with whatever labels you can get (a few confirmed cases) and tune the alert rate to what reviewers can handle. → *More:* Ch 9 §9.7 — Anomaly detection

Q117. How do you evaluate a clustering when you have no labels? Internal metrics: silhouette score, Davies–Bouldin index (lower is better), Calinski–Harabasz. Stability: do clusters stay the same when you resample or change the seed? Usefulness: can a domain expert name them, and do they improve a downstream task (targeting, features for a model)? If some labels exist, use adjusted Rand index or normalised mutual information. → *More:* Ch 9 §9.9 — Evaluating unsupervised models

Q118. What are association rules (support, confidence, lift)? Used in market-basket analysis ("people who buy bread and butter also buy milk"). Support: how often the item set appears. Confidence: P(milk | bread and butter). Lift: confidence divided by milk's overall frequency; lift above 1 means the items co-occur more than by chance. Apriori finds frequent item sets by pruning any set whose subsets are not frequent. → *More:* Ch 9 §9.8 — Association rules

---

9. Probability, statistics and math

Q119. ⋆ Explain Bayes' theorem with an example. P(A | B) = P(B | A) · P(A) / P(B): update a prior belief P(A) with evidence B. Example: a disease affects 1% of people; a test catches 90% of cases and gives 5% false positives. P(positive) = 0.9 × 0.01 + 0.05 × 0.99 = 0.0585, so P(sick | positive) = 0.009 / 0.0585 ≈ 15%. Most positives are false because the disease is rare: the base-rate effect, the same reason rare-event classifiers have low precision. → *More:* Ch 2 §2.7 — Conditional probability and Bayes' theorem

Q120. ⋆ What is a p-value? What is it not? The p-value is the probability of seeing data at least as extreme as what you observed if the null hypothesis (no effect) were true. A small p-value (below a pre-chosen α such as 0.05) means the data would be surprising under no effect. It is not the probability that the null hypothesis is true, not the size or importance of the effect, and it becomes misleading if you test many things or peek repeatedly and stop when significant. → *More:* Ch 2 §2.11 — Hypothesis tests, p-values and confidence intervals

Q121. ⋆ What is a confidence interval? A range computed from the sample by a procedure that, over many repeated samples, contains the true value a stated fraction of the time (for example 95%). It does not mean "there is a 95% chance the true value is in this particular interval" (that is the Bayesian credible interval's meaning). Its width shows the precision of the estimate; it shrinks roughly with 1 / √n. → *More:* Ch 2 §2.11

Q122. ⋆ What is the central limit theorem and why does it matter? The average of many independent samples from almost any distribution with finite variance is approximately normally distributed, with standard error σ / √n, however skewed the original data. It is why z-tests and t-tests, confidence intervals and A/B-test significance calculations work on non-normal metrics like revenue per user, given enough samples. → *More:* Ch 2 §2.10 — Sampling and the central limit theorem

Q123. What is the normal distribution and why do we care? How do you check normality? The bell curve, defined by mean and standard deviation; about 68%, 95% and 99.7% of values fall within 1, 2 and 3 standard deviations. It appears through the central limit theorem and is the assumption behind many tests and the MSE loss (which is maximum likelihood under Gaussian noise). Check normality with a histogram, a Q-Q plot (points on a straight line), or tests such as Shapiro–Wilk (which flag tiny deviations on large samples). → *More:* Ch 2 §2.6 — Random variables and distributions

Q124. Name common probability distributions and when each appears. Bernoulli (one yes/no event, a click); binomial (number of successes in n trials); Poisson (count of events in a fixed interval, such as orders per minute); exponential (time between Poisson events); normal (sums and averages); uniform (equal chance); log-normal (incomes, prices); beta (a distribution over probabilities, used for click-rate priors and Thompson sampling). → *More:* Ch 2 §2.6

Q125. ⋆ Correlation vs causation? Correlation means two variables move together; causation means changing one changes the other. Correlation can come from a confounder (ice-cream sales and drownings both rise in summer), reverse causation or chance. To establish causation you need a randomised experiment (A/B test) or careful causal-inference methods. Predictive models exploit correlation, which is fine for prediction but risky when you use the model to decide on an intervention. → *More:* Ch 2 §2.9 — Correlation vs causation

Q126. Pearson vs Spearman correlation? Pearson measures linear association between two numeric variables and is sensitive to outliers. Spearman is Pearson computed on ranks, so it measures any monotonic relationship and is robust to outliers. Both range from −1 to 1; zero correlation does not mean independence (y = x² over symmetric x has Pearson ≈ 0). → *More:* Ch 2 §2.9

Q127. ⋆ What is maximum likelihood estimation (MLE)? How does it relate to common losses? MLE picks the parameters that make the observed data most probable. Minimising MSE is MLE under Gaussian noise; minimising log loss is MLE for Bernoulli/categorical outcomes. MAP (maximum a posteriori) estimation adds a prior over parameters: a Gaussian prior gives L2 regularisation and a Laplace prior gives L1. → *More:* Ch 2 §2.12, Ch 5 §5.8

Q128. Bayesian vs frequentist statistics? Frequentists treat parameters as fixed unknowns and probability as long-run frequency; they use p-values and confidence intervals. Bayesians treat parameters as random with a prior distribution and update it to a posterior with data, giving direct

statements like "P(B is better than A) = 92%". Bayesian methods handle small data and prior knowledge naturally but require choosing priors and can be costlier to compute.

Q129. ⋆ What are entropy, cross-entropy and KL divergence? Entropy H(p) = −Σ p log p measures the uncertainty of a distribution (a fair coin has 1 bit). Cross-entropy H(p, q) = −Σ p log q is the average cost of encoding outcomes from p using a code built for q; with one-hot labels it is exactly the classification loss. KL divergence KL(p‖q) = H(p, q) − H(p) measures how far q is from p; it is non-negative and not symmetric. KL appears in VAEs, knowledge distillation and RLHF penalties. → *More:* Ch 2 §2.12 — Information theory

Q130. ⋆ Explain gradient descent. What happens if the learning rate is too large or too small? Gradient descent repeatedly moves parameters a small step against the gradient (the direction of steepest increase of the loss): w ← w − η·∇L(w), where η is the learning rate. Too small: very slow progress, may stall. Too large: overshoots the minimum, oscillates or diverges (loss goes to infinity or NaN). Feature scaling makes the loss surface rounder so one learning rate suits all directions. → *More:* Ch 2 §2.5 — Gradient descent, Ch 5 §5.4

Q131. ⋆ Batch vs stochastic vs mini-batch gradient descent? Batch gradient descent computes the gradient on the full dataset per step: accurate but slow and memory-hungry. Stochastic (SGD) uses one example per step: cheap and noisy; the noise can help escape poor regions but makes convergence jumpy. Mini-batch (32–1024 examples) is the practical default: GPU-friendly vectorised maths with moderate noise. An *epoch* is one pass through all data; an *iteration* is one update. → *More:* Ch 5 §5.4 — Gradient descent in practice

Q132. What is a convex function, and why does convexity matter in ML? A function is convex if the straight line between any two points on its graph lies on or above the graph, so any local minimum is the global minimum. Linear regression with MSE, logistic regression with log loss and SVMs are convex, so gradient descent reliably finds the best solution. Neural network losses are non-convex, yet in practice gradient descent finds good-enough minima. → *More:* Ch 2 §2.5

Q133. What are eigenvalues and eigenvectors? Where do they appear in ML? An eigenvector v of a square matrix A is a direction that A only stretches: Av = λv, where λ (the eigenvalue) is the stretch factor. In PCA the eigenvectors of the covariance matrix are the principal components and the eigenvalues are the variance along each. They also appear in PageRank, spectral clustering and the analysis of exploding/vanishing gradients in RNNs. → *More:* Ch 2 §2.3 — Eigenvalues, eigenvectors and SVD

Q134. What are the gradient, Jacobian and Hessian? The gradient is the vector of partial derivatives of a scalar function, pointing uphill. The Jacobian is the matrix of partial derivatives of a vector-valued function (each output with respect to each input), used when chaining layers in backpropagation. The Hessian is the matrix of second derivatives, describing curvature; Newton's method uses it, and XGBoost uses its diagonal. → *More:* Ch 2 §2.4 — Derivatives, partial derivatives and gradients

Q135. Variance, standard deviation and standard error: what's the difference? Variance is the average squared distance from the mean; standard deviation is its square root, in the data's units, describing the spread of individual values. Standard error is the standard deviation of an estimate (such as a sample mean), σ / √n, describing how precisely you know it. More data shrinks the standard error, not the standard deviation. → *More:* Ch 2 §2.8 — Descriptive statistics

Q136. Probability puzzle: a family has two children and at least one is a girl. What is the probability both are girls? List equally likely outcomes: GG, GB, BG, BB. "At least one girl" removes BB, leaving three, of which one is GG, so the answer is 1/3. (If instead you know the *older* child is a girl, the answer is 1/2.) The point is to condition on exactly the information given. → *More:* Ch 2 §2.7

Q137. Can two events with P(A) = 0.6 and P(B) = 0.7 be disjoint? No. Disjoint (mutually exclusive) events satisfy P(A or B) = P(A) + P(B) = 1.3, which exceeds 1, impossible. They must overlap by at least 0.3: P(A and B) ≥ 0.6 + 0.7 − 1.

Q138. What is the expected number of coin flips to get two heads in a row? Let E be the expected flips from the start. Flip once: tails (½) restarts; heads (½) then a second flip either finishes (½) or restarts (½). E = ½(1 + E) + ½·½·2 + ½·½(2 + E), which gives E = 6. Interviewers want the method: set up states and solve linear equations for expectations.

Q139. ⋆ How do you decide the sample size for an A/B test? What is statistical power? Power is the probability of detecting a real effect of a given size (commonly 80%). Sample size per group grows with

the metric's variance and shrinks with the square of the minimum detectable effect (MDE): roughly $n \approx 16 \cdot \sigma^2 / \delta^2$ per group for $\alpha = 0.05$ and 80% power. Halving the effect you want to detect needs four times the traffic. Decide the size before starting and do not stop early when the result first looks significant. → *More:* Ch 16 §16.9 — Online experiments and A/B testing

Q140. What is the multiple-comparisons problem? Test 20 independent metrics at $\alpha = 0.05$ and you expect one "significant" result by pure chance. Corrections: Bonferroni (divide α by the number of tests; conservative), Holm, or Benjamini–Hochberg (controls the false discovery rate). Better still, name one primary metric before the experiment. → *More:* Ch 2 §2.11

Q141. How would you sample uniformly from a stream of unknown length (reservoir sampling)? Keep the first k items. For the i-th item (counting from 1, $i > k$), draw a random integer j from 1 to i; if $j \leq k$, replace slot j with the new item. So the new item enters with probability k / i and evicts a uniformly random current member. By induction, after n items every item is in the reservoir with probability exactly k / n. It uses $O(k)$ memory and one pass, which is useful for sampling training data from logs.

---

10. Deep learning fundamentals

Q142. How does a feed-forward neural network work? Inputs pass through layers; each layer computes a weighted sum of the previous layer's outputs plus a bias and applies a non-linear activation function; the final layer produces the prediction (a number, or class probabilities via softmax). Training compares the prediction with the label using a loss, computes gradients with backpropagation, and updates the weights with an optimizer. Stacked layers learn increasingly abstract features. → *More:* Ch 10 §10.1 — From the perceptron to the MLP

Q143. ⋆ Explain backpropagation. Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule backwards through the network. The forward pass stores each layer's intermediate values; the backward pass starts from $\partial \text{loss} / \partial \text{output}$ and, layer by layer, multiplies by each layer's local derivative, reusing results so the whole gradient costs only a small constant multiple of a forward pass (roughly two forward passes' worth of compute). An optimizer such as SGD or Adam then uses the gradients to update the weights. Frameworks do this automatically (autograd). → *More:* Ch 10 §10.4 — Backpropagation

Q144. ⋆ Why do we need non-linear activation functions? A composition of linear layers is just one linear function, so without non-linearities a 100-layer network is no more powerful than logistic regression. Non-linear activations let networks represent curved decision boundaries and complex functions; with enough hidden units, a network with one hidden layer can approximate any continuous function (the universal approximation theorem), though depth makes this far more efficient. → *More:* Ch 10 §10.2 — Activation functions

Q145. ⋆ Compare sigmoid, tanh, ReLU, Leaky ReLU and GELU. Sigmoid squashes to (0, 1), saturates at both ends (gradients near zero) and is not zero-centred: use only for binary output probabilities or gates. Tanh squashes to (−1, 1), zero-centred, still saturates. ReLU, $\max(0, x)$, is cheap, does not saturate for positive inputs and is the default for hidden layers, but units can "die" (stuck at zero). Leaky ReLU keeps a small slope for negatives to avoid dead units. GELU and SiLU/Swish are smooth ReLU-like functions used in transformers. → *More:* Ch 10 §10.2

Q146. What is the dying ReLU problem? If a ReLU unit's input becomes negative for all examples (often after a large update from a high learning rate), its output and gradient are zero, so it never updates again. Fixes: a lower learning rate, Leaky ReLU/ELU/GELU, careful (He) initialisation, and normalisation layers. → *More:* Ch 10 §10.2

Q147. ⋆ What are vanishing and exploding gradients? How do you fix them? During backpropagation the gradient for early layers is a product of many per-layer factors. If those factors are mostly below 1 the gradient shrinks exponentially (vanishing) and early layers stop learning; the sigmoid's derivative is at most 0.25, so deep sigmoid networks vanish easily; if above 1 it grows exponentially (exploding) and training diverges. Fixes: ReLU-family activations, good initialisation (He/Xavier), batch or layer normalisation, residual (skip) connections, gated RNNs (LSTM/GRU), and gradient clipping for explosions. → *More:* Ch 10 §10.6 — Initialization and vanishing/exploding gradients

Q148. ⋆ What happens if you initialise all weights to zero? How should you initialise? Every neuron in a layer computes the same output and receives the same gradient, so they stay identical forever (the symmetry is never broken) and the layer acts like one neuron. Initialise randomly with a scale that keeps the variance of activations stable across layers: Xavier/Glorot (variance ∝ 1/fan-in, or 2/(fan-in + fan-out)) for tanh/sigmoid, He (variance 2/fan-in) for ReLU. Biases can start at zero. → *More:* Ch 10 §10.6

Q149. ⋆ Explain SGD with momentum, RMSProp and Adam. When would you use SGD instead of Adam? Momentum keeps a running average of past gradients (a velocity), smoothing noise and speeding progress along consistent directions. RMSProp divides each parameter's step by a running average of its squared gradients, giving per-parameter learning rates. Adam combines both, with bias correction for the early steps; it converges fast with little tuning and is the default for transformers. SGD with momentum often generalises slightly better for CNNs on vision tasks, given a good learning-rate schedule. → *More:* Ch 10 §10.5 — Optimizers and learning-rate schedules

Q150. What is AdamW and why is it preferred over Adam with L2? In Adam, an L2 penalty added to the loss gets divided by the adaptive denominator, so weights with large gradients are barely regularised. AdamW applies weight decay directly to the weights, separately from the gradient update (decoupled weight decay), which regularises uniformly and generalises better. It is the standard optimizer for training transformers. → *More:* Ch 10 §10.5

Q151. Why use a learning-rate schedule? What is warm-up? A larger learning rate early makes fast progress; a smaller one later settles into a good minimum. Common schedules: step decay, cosine decay and one-cycle. Warm-up starts with a tiny learning rate and ramps up over the first steps, which avoids unstable early updates when Adam's statistics and normalisation layers are not yet reliable; it is standard for transformers. A learning-rate range test (increase the rate until the loss blows up) helps pick the peak. → *More:* Ch 10 §10.5

Q152. ⋆ What is dropout and how does it behave at training vs inference time? During training, dropout sets each unit's output to zero with probability p (for example 0.1–0.5) and scales the survivors by 1/(1 − p), so the network cannot rely on any single co-adapted unit; it acts like training an ensemble of thinned networks. At inference dropout is switched off (`model.eval()` in PyTorch) and all units are used. Forgetting to switch modes is a classic bug. → *More:* Ch 10 §10.7 — Regularization

Q153. ⋆ What is batch normalisation and why does it help? Batch norm vs layer norm? Batch norm normalises each feature (each channel in a CNN, over the batch and spatial positions) to mean 0 and variance 1 over the current mini-batch, then applies a learned scale and shift. It stabilises training, allows higher learning rates, reduces sensitivity to initialisation and adds mild regularisation (batch statistics are noisy). The original "internal covariate shift" explanation is disputed; a smoother loss landscape is the more accepted reason it helps. At inference it uses running averages collected in training. Layer norm normalises across the features of each single example instead, so it does not depend on batch size, which is why transformers and RNNs use it. → *More:* Ch 10 §10.7

Q154. Which loss functions do you use for which tasks? Regression: MSE (or MAE/Huber for outliers). Binary classification: binary cross-entropy on a sigmoid output (in PyTorch use `BCEWithLogitsLoss` on raw logits). Multiclass: categorical cross-entropy on softmax (`CrossEntropyLoss` takes logits). Multilabel: independent sigmoids with BCE. Ranking: pairwise or listwise losses. Embeddings: contrastive or triplet loss. Imbalanced detection: weighted cross-entropy or focal loss. → *More:* Ch 10 §10.3 — The forward pass and loss functions, Ch 20 §20.6

Q155. Epoch vs batch vs iteration? A batch is the group of examples used for one gradient update; an iteration (step) is one update; an epoch is one full pass over the training data. With 10,000 examples and batch size 100, one epoch is 100 iterations. → *More:* Ch 10 §10.8 — Practical training habits

Q156. How does batch size affect training? Larger batches give less noisy gradients and better hardware use, but need more memory, often need a proportionally larger learning rate (with warm-up), and can generalise slightly worse. Smaller batches add noise that can act as regularisation but make training slower per example. When memory is short, use gradient accumulation to simulate a larger batch. → *More:* Ch 10 §10.8

Q157. ⋆ Your network's training loss is not decreasing. How do you debug it? First try to overfit a tiny batch (10 examples): if it cannot reach near-zero loss, there is a bug. Check: data and labels line up (plot some), inputs are normalised, the loss matches the output layer (logits vs probabilities,

label format), the learning rate (try ten times smaller and larger), gradients are flowing (no `detach`, `zero_grad` called each step, no dead ReLUs, no NaNs), and `model.train()`/`model.eval()` modes. Then add complexity back one step at a time. → *More:* Ch 10 §10.8

Q158. Your loss becomes NaN during training. What could cause it? A learning rate that is too high (exploding updates); exploding gradients (clip them); log of zero or division by zero in a hand-written loss (use the library's numerically stable versions that take logits); bad input values (NaN or infinite features, unnormalised huge values); or mixed-precision overflow (use a gradient scaler or bfloat16). → *More:* Ch 10 §10.6, Ch 10 §10.8

Q159. What are residual (skip) connections and why do they matter? A residual block outputs x + F(x): the layers learn only a correction to the identity. Gradients flow straight through the addition, so very deep networks (100+ layers) train without vanishing gradients, and adding a layer can never make things much worse because it can learn F ≈ 0. Introduced with ResNet; every transformer layer uses them. → *More:* Ch 11 §11.4 — Famous architectures

Q160. What is an autoencoder? Give applications. A network that compresses the input into a small bottleneck vector (encoder) and reconstructs it (decoder), trained to minimise reconstruction error. Uses: dimensionality reduction, denoising, anomaly detection (high reconstruction error means unusual), and pretraining. Variational autoencoders (VAEs) make the bottleneck a probability distribution so you can sample new data. → *More:* Ch 9 §9.7

Q161. What are the components of a GAN, and what makes GANs hard to train? A generative adversarial network has a generator (turns random noise into fake samples) and a discriminator (tells real from fake); they train in a two-player game until fakes are indistinguishable. Training is unstable: mode collapse (the generator produces little variety), oscillation, and one side overpowering the other. Diffusion models, which learn to remove noise step by step, have largely replaced GANs for image generation because they train more stably.

Q162. What is knowledge distillation? Train a small "student" model to match the softened output probabilities of a large "teacher" model (softmax with a temperature above 1), usually alongside the true labels. The soft targets carry information about which wrong classes are similar, so the student learns more than from hard labels. It is used to make models small and fast enough for serving. → *More:* Ch 16 §16.6 — Model serving basics

Q163. How do you make a model smaller or faster for inference? Quantisation (store weights and compute in 8-bit or 4-bit integers instead of 32-bit floats), pruning (remove small weights or whole channels), knowledge distillation, efficient architectures, batching requests, caching, compiling the model (`torch.compile`, ONNX Runtime, TensorRT) and running on suitable hardware. Measure the accuracy loss on your own validation set after each step. → *More:* Ch 16 §16.6

Q164. What is the difference between `model.train()` / `model.eval()` and `torch.no_grad()` in PyTorch? `model.train()` and `model.eval()` switch layer behaviour: dropout on/off and batch norm using batch statistics vs running statistics. `torch.no_grad()` (or `torch.inference_mode()`) stops autograd from recording operations, saving memory and time. For inference you want both: `model.eval()` and a no-grad context. → *More:* Ch 10 §10.9 — PyTorch basics

---

11. CNNs, RNNs, attention and transformers

Q165. How does a convolutional layer work? What determines the output shape? A small filter (kernel), say 3×3 across all input channels, slides over the image; at each position it takes a weighted sum of the patch, producing one value of an output feature map. A layer learns many filters, one output channel each. For a grayscale input the filter is 3×3×1; for RGB it is 3×3×3. Output width = ⌊(W − K + 2P) / S⌋ + 1 for input width W, kernel K, padding P and stride S; output depth = number of filters. → *More:* Ch 11 §11.1 — Convolution

Q166. ⋆ Why use convolutions for images instead of fully connected layers? Weight sharing: the same filter is applied everywhere, so a 3×3 filter has 9 weights per channel instead of millions of connections. Local connectivity matches the fact that nearby pixels are related. Translation equivariance: a pattern is detected wherever it appears. This means far fewer parameters, less

overfitting and better generalisation; stacking layers grows the receptive field from edges to textures to objects. → *More:* Ch 11 §11.1

Q167. What is pooling and why use it? Are CNNs invariant to rotation? Pooling downsamples feature maps: max pooling keeps the strongest activation in each window, average pooling the mean. It reduces computation, enlarges the receptive field and adds a little translation invariance. CNNs are not naturally invariant to rotation or scale: a rotated image can change the prediction, which is why rotation and scale augmentations are used in training. → *More:* Ch 11 §11.2 — Pooling and the CNN architecture

Q168. What is a receptive field, and why use several 3×3 kernels instead of one large kernel? The receptive field is the region of the input that affects one output unit. Two stacked 3×3 convolutions see 5×5 and three see 7×7, with fewer parameters (3 × 9 = 27 vs 49 per channel pair) and more non-linearities in between, so the network is both cheaper and more expressive. This was the key insight of VGG. → *More:* Ch 11 §11.2, Ch 11 §11.4

Q169. Name famous CNN architectures and their main idea. LeNet (early small CNN for digits); AlexNet (deep CNN on GPUs with ReLU and dropout, won ImageNet 2012); VGG (stacks of 3×3 convolutions); GoogLeNet/Inception (parallel filter sizes); ResNet (skip connections enable very deep nets); MobileNet/EfficientNet (efficient convolutions and scaling for mobile); Vision Transformers (ViT) treat image patches as tokens. → *More:* Ch 11 §11.4 — Famous architectures

Q170. ⋆ What is transfer learning and how do you fine-tune a pretrained model? Reuse a model pretrained on a large dataset (ImageNet, a large text corpus) whose early layers already detect general features. Replace the output head with one for your classes; first train only the head with the backbone frozen, then optionally unfreeze some or all layers and fine-tune with a small learning rate. It gives strong results with few labels and little compute. → *More:* Ch 11 §11.5 — Transfer learning and fine-tuning

Q171. Classification vs object detection vs segmentation? What are IoU and non-maximum suppression? Classification gives one label per image; detection gives labelled bounding boxes (Faster R-CNN, YOLO); semantic segmentation labels every pixel with a class, and instance segmentation separates individual objects (Mask R-CNN). IoU (intersection over union) is the overlap of two boxes divided by their combined area, used to match predictions to ground truth. Non-maximum suppression keeps the highest-scoring box and removes others that overlap it above an IoU threshold. → *More:* Ch 11 §11.7 — Vision tasks

Q172. ⋆ How does an RNN work, and why is it hard to train on long sequences? A recurrent neural network processes a sequence one step at a time, updating a hidden state $h_t = f(W \cdot h_{t-1} + U \cdot x_t)$ that carries information forward. Training unrolls the network over time (backpropagation through time), so gradients are products over many steps and vanish or explode; the network then forgets long-range dependencies. It is also sequential, so it cannot be parallelised across time steps. → *More:* Ch 12 §12.2–12.3 — RNNs and vanishing gradients through time

Q173. ⋆ How do LSTMs and GRUs solve the vanishing-gradient problem? An LSTM adds a cell state updated additively, $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$, controlled by sigmoid gates: forget (f), input (i) and output. Because the update is additive and the forget gate can stay near 1, information and gradients can flow across many steps. A GRU merges the gates into update and reset gates, with fewer parameters and similar accuracy. → *More:* Ch 12 §12.4 — LSTM and GRU

Q174. What is a sequence-to-sequence (encoder–decoder) model? An encoder reads the input sequence into a representation and a decoder generates the output sequence token by token, used for translation, summarisation and speech recognition. Early versions squeezed the whole input into one fixed vector, a bottleneck for long inputs; attention removed it by letting the decoder look back at all encoder states. During training the decoder is fed the true previous token (teacher forcing). → *More:* Ch 12 §12.5 — Sequence-to-sequence models

Q175. ⋆ Explain the attention mechanism. What are Q, K and V? Attention lets each position build its output as a weighted average of other positions' information, with weights computed from similarity. Each token produces a query (what I am looking for), a key (what I contain) and a value (what I pass on). Scores are the dot products of a query with all keys; softmax turns them into weights; the output is the weighted sum of values: $\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k}) \cdot V$. → *More:* Ch 12 §12.6 — The attention mechanism

Q176. ⋆ Why divide by √d_k in scaled dot-product attention? For random vectors of dimension d_k, dot products have variance proportional to d_k, so with large dimensions the scores become large and softmax saturates: one weight near 1, the rest near 0, with tiny gradients. Dividing by √d_k keeps the scores at unit scale so softmax stays smooth and trainable. → *More:* Ch 12 §12.6

Q177. ⋆ Explain the transformer architecture. A stack of identical blocks, each with multi-head self-attention (every token attends to every other token, in several heads that can learn different relations) followed by a position-wise feed-forward network, both wrapped with residual connections and layer normalisation. Positional encodings add order information, since attention itself ignores order. Encoder-only models (BERT) read text bidirectionally; decoder-only models (GPT-style) use causal masking so each token sees only earlier ones and generate text; encoder–decoder models (T5) translate between sequences. → *More:* Ch 12 §12.7 — Self-attention and the transformer

Q178. Why did transformers replace RNNs? What's the downside? Transformers process all positions in parallel (fast training on GPUs) and connect any two tokens in one step, so long-range dependencies are easy to learn; RNNs process tokens one after another and pass information through many steps. The downside: self-attention costs $O(n^2)$ time and memory in the sequence length n, which is why long-context models use tricks such as FlashAttention, sparse or sliding-window attention and KV caching. → *More:* Ch 12 §12.7

Q179. Why do transformers need positional encoding? Self-attention has no built-in notion of order: if you permute the input tokens, the outputs are just permuted the same way, so without extra information "dog bites man" and "man bites dog" look alike. Positional encodings (fixed sinusoids, learned position embeddings, or rotary embeddings, RoPE, which rotate queries and keys by position) inject order. → *More:* Ch 12 §12.7

Q180. What is multi-head attention? Instead of one attention with large vectors, the model projects Q, K and V into h smaller subspaces and runs attention in each head in parallel, then concatenates and projects the results. Different heads can specialise (one tracks syntax, another coreference) at the same total cost as a single full-size head. → *More:* Ch 12 §12.7

Q181. ⋆ What are word embeddings? How does word2vec work? Embeddings are dense vectors (say 100–1,000 numbers) for words, learned so that words used in similar contexts get similar vectors, enabling analogies like king − man + woman ≈ queen. Word2vec trains a shallow network either to predict a word from its neighbours (CBOW) or neighbours from a word (skip-gram), using negative sampling to keep it cheap. Static embeddings give one vector per word; transformer models give contextual embeddings that differ by sentence ("bank" of a river vs a bank account). → *More:* Ch 12 §12.9 — Word embeddings

Q182. What is TF-IDF and bag-of-words? What are their limitations? Bag-of-words represents a document by word counts, ignoring order. TF-IDF weights each count by inverse document frequency, log(N / number of documents containing the word), so common words like "the" count little and distinctive words count a lot. Adding n-grams (word pairs) captures some order. Limitations: huge sparse vectors, no notion of synonyms, and no word order beyond n-grams. Still a strong, fast baseline with logistic regression. → *More:* Ch 12 §12.8 — Text preprocessing and tokenization

Q183. What is tokenization? Stemming vs lemmatization? Why subword tokenizers? Tokenization splits text into units (tokens). Stemming chops word endings by rules ("studies" → "studi"); lemmatization maps to the dictionary form using vocabulary and grammar ("studies" → "study"). Modern models use subword tokenizers (BPE, WordPiece, SentencePiece) that split rare words into frequent pieces, so the vocabulary stays fixed-size yet any word, typo or name can be represented without "unknown" tokens. → *More:* Ch 12 §12.8

Q184. BERT vs GPT: what's the difference? BERT is an encoder-only transformer pretrained by masked-language modelling (predict hidden words using context on both sides); it is good for understanding tasks such as classification, entity tagging and search embeddings. GPT is decoder-only, pretrained to predict the next token left to right; it is good at generation and, at scale, at following instructions and in-context learning. → *More:* Ch 12 §12.7

Q185. What is retrieval-augmented generation (RAG)? Before a language model answers, the system retrieves relevant documents (usually by embedding the question and searching a vector index, often combined with keyword search) and puts them into the prompt, so the model answers from current, private or citable sources instead of only its training memory. Quality depends on chunking, the embedding

model, re-ranking and evaluation of both retrieval and the final answer. → *More:* Ch 13 §13.9 — ANN search, Ch 17 §17.4

Q186. How would you forecast a time series? How is it different from ordinary regression? Observations are ordered and correlated over time, often with trend and seasonality, so you must split by time and avoid features that peek into the future. Baselines: last value, seasonal naive (same day last week), moving averages. Classical models: exponential smoothing, ARIMA. ML: gradient boosting on lag and rolling-window features plus calendar features; deep models (LSTMs, temporal transformers) for many related series. Evaluate with rolling-origin backtests. → *More:* Ch 12 §12.10 — Time-series forecasting

Q187. How do RLHF and reinforcement learning relate to large language models? After pretraining and supervised fine-tuning on example answers, RLHF (reinforcement learning from human feedback) collects human preference rankings between answers, trains a reward model on them, then optimises the language model to maximise that reward (for example with PPO), with a KL penalty that keeps it close to the original model. DPO (direct preference optimisation) skips the separate reward model and trains on preference pairs directly. → *More:* Ch 14 §14.10 — RLHF

Q188. Explore vs exploit: what is a multi-armed bandit, and Q-learning vs SARSA? A bandit repeatedly chooses among options with unknown payoffs, balancing trying options (explore) against using the best so far (exploit), with ε-greedy, UCB (optimism under uncertainty) or Thompson sampling (sample from the posterior). In full reinforcement learning, Q-learning updates toward the best next action's value (off-policy, learns the optimal policy), while SARSA updates toward the action actually taken (on-policy, safer while exploring). → *More:* Ch 14 §14.4, Ch 14 §14.7

---

12. Recommender systems and embeddings

Q189. ⋆ Content-based vs collaborative filtering? Content-based filtering recommends items similar to what the user liked, using item attributes (genre, text, image embeddings); it works for new items but tends to recommend more of the same. Collaborative filtering uses the behaviour of many users ("people who liked X also liked Y"), discovering surprising, high-quality items without knowing what they are, but it fails for new users and items with no interactions. Production systems are hybrids. → *More:* Ch 13 §13.4 — Content-based vs collaborative filtering

Q190. Explicit vs implicit feedback? How do you use implicit feedback? Explicit feedback is a direct rating (stars, thumbs up), scarce but clear. Implicit feedback is behaviour (clicks, watch time, purchases, skips), plentiful but noisy: a missing interaction may mean "not seen" rather than "disliked", and clicks are biased by position and by what the old system showed. Use implicit signals as weighted positives (longer watch time = stronger), sample negatives from items shown but not engaged, and correct for position bias. → *More:* Ch 13 §13.2 — Explicit vs implicit feedback

Q191. How does matrix factorisation work for recommendations? Represent the sparse user–item interaction matrix R as the product of two low-rank matrices: each user gets a vector u_i and each item a vector v_j (say 64 numbers), and the predicted affinity is their dot product $u_i \cdot v_j$. Learn the vectors by minimising the error on observed entries plus L2 regularisation, with SGD or alternating least squares (ALS: fix items, solve users exactly, then swap). The learned vectors are embeddings in which similar users and items end up close. → *More:* Ch 13 §13.5 — Matrix factorisation

Q192. ⋆ What is the cold-start problem and how do you handle it? New users or items have no interaction history, so collaborative methods cannot score them. For new users: popular or trending items, onboarding questions, context (location, device, time), and fast session-based updates. For new items: content features (text, image and metadata embeddings) mapped into the same space, plus controlled exploration (a small share of traffic, bandits) to gather interactions quickly. → *More:* Ch 13 §13.10 — Cold start

Q193. What is a two-tower model and how is it used for candidate retrieval? One neural network (tower) embeds the user and context, another embeds the item, and the score is the dot product of the two vectors. Because items do not depend on the user, all item vectors are computed offline and loaded into an approximate-nearest-neighbour index; at request time only the user tower runs, and the index returns the top few hundred items in milliseconds. Training uses positives from engagement and in-batch

negatives (other items in the batch), with a correction so popular items are not over-penalised. → *More:* Ch 13 §13.7 — Two-tower retrieval

Q194. Why do recommenders use a multi-stage funnel (retrieval → ranking → re-ranking)? Scoring hundreds of millions of items with a heavy model per request is impossible within a ~100 ms budget. Retrieval cheaply narrows to hundreds or thousands of candidates from several sources (two-tower, co-visitation, trending); a ranking model with rich features and cross features scores those precisely; re-ranking applies business rules, diversity, freshness, de-duplication and policy filters. Each stage trades accuracy for speed differently. → *More:* Ch 13 §13.8 — The funnel

Q195. What is approximate nearest-neighbour (ANN) search? Name some methods. Exact nearest-neighbour search over millions of vectors is too slow, so ANN indexes accept a small loss in recall for huge speed-ups. HNSW builds a multi-layer graph and walks greedily toward the query; IVF clusters vectors and searches only the nearest clusters; product quantisation compresses vectors to save memory. Libraries: FAISS, ScaNN, and vector databases. Tune for recall@k against latency. → *More:* Ch 13 §13.9 — ANN search

Q196. How do you evaluate a recommender system offline and online? Offline: split by time, then measure recall@k and NDCG@k (ranking quality), plus coverage (share of the catalogue ever recommended), diversity and novelty. Online: A/B test the business metrics (watch time, purchases, retention, long-term satisfaction) with guardrails (latency, complaints). Offline gains often shrink online because logged data reflects the old system's choices, so treat offline metrics as a filter, not the verdict. → *More:* Ch 13 §13.11 — Evaluation

Q197. What are popularity bias and feedback loops in recommenders? How do you counter them? The system shows popular items, they collect more clicks, the next model learns they are even better, and the rich get richer while niche items and new creators never get exposure. Counter with exploration traffic, diversity constraints in re-ranking, debiasing (inverse propensity weighting of logged data), and metrics such as coverage and long-tail share. → *More:* Ch 13 §13.12 — Popularity bias and feedback loops

Q198. What are good similarity measures for embeddings? Cosine vs dot product vs Euclidean? Cosine similarity compares only direction (it ignores vector length), good for text embeddings. Dot product also rewards length, which models can use to encode popularity or confidence; two-tower retrieval often uses it. Euclidean distance is equivalent to cosine for normalised vectors. Jaccard similarity suits sets (shared items between two users). → *More:* Ch 13 §13.3 — Similarity measures

---

13. ML system design

Q199. ⋆ How do you approach an ML system design interview? What framework do you use? Use a repeatable structure and say it out loud: (1) clarify the goal and frame it as an ML task (inputs, output, what decision it drives); (2) define success: an offline metric and the online business metric plus guardrails; (3) data and labels: sources, how labels are made, volumes; (4) features and a baseline model, then a stronger model; (5) serving: batch vs online, latency budget, the multi-stage funnel if catalogues are large; (6) evaluation, A/B testing, monitoring and retraining; (7) failure modes, fairness and cost. Spend most time where the problem is hardest, and state assumptions with napkin numbers. → *More:* Ch 15 §15.3 — The seven-step framework

Q200. How does ML system design differ from classic system design? Classic design focuses on storage, APIs, scaling and consistency. ML design adds data and labels as first-class components, a training pipeline separate from the serving path, an objective that must be translated from business goals, offline and online evaluation, and the fact that a system can silently degrade as data drifts while every server stays healthy. Interviewers expect you to talk about metrics, labels and feedback loops, not just boxes. → *More:* Ch 15 §15.1 — ML design vs classic system design

Q201. ⋆ Design a video (or product) recommendation system. Goal: maximise long-term satisfaction, measured online by watch time, retention and explicit feedback, not just clicks (which reward clickbait). Data: impressions, watches with watch fraction, likes, skips, search history, item metadata. Architecture: candidate generation from several sources (two-tower embeddings with an ANN index, co-watch, subscriptions, trending) → a ranker (gradient-boosted trees or a deep multi-task network predicting click, watch time and like, combined in a weighted score) → re-ranking for

diversity, freshness and policy. Cold start via content embeddings and exploration. Offline recall@k/NDCG on a time split; online A/B test; monitor drift and popularity bias; retrain daily, with near-real-time features for session context. → *More:* Ch 17 §17.1 — Video recommendations

Q202. ⋆ Design a news-feed ranking system. Rank candidate posts from friends, groups and pages for each user. Predict several engagement probabilities (click, like, comment, share, hide/report) with a multi-task model and combine them into a value score with tuned weights, including negative terms for hides and reports. Features: author affinity, post type and age, past engagement, content embeddings. Re-rank for diversity and integrity. Watch for engagement-bait, position bias in labels and long-term effects; evaluate with A/B tests on meaningful-interaction metrics and long-run retention. → *More:* Ch 17 §17.2 — Social feed ranking

Q203. ⋆ Design a search ranking system (e-commerce or web search). Two stages: retrieval (keyword/BM25 and embedding-based semantic search, merged) then learning-to-rank on the top few hundred results. Labels from clicks, add-to-cart and purchases (corrected for position bias, for example with randomised position experiments or click models), plus human relevance judgements. Features: query–document text match, embedding similarity, item quality and popularity, personalisation, price. Losses: pointwise, pairwise (RankNet) or listwise (LambdaMART). Offline NDCG@10 and MRR; online conversion and search abandonment; interleaving experiments detect ranking differences with less traffic. → *More:* Ch 17 §17.3 — E-commerce search ranking

Q204. ⋆ Design an ad click-through-rate (CTR) prediction system. Predict P(click | user, ad, context) for every ad in an auction; expected value = bid × pCTR, so predictions must be calibrated, not just ranked. Data is huge (billions of impressions/day) with rare positives (~1%), very sparse categorical features (user, ad, publisher IDs) and fast-changing ads. Models: logistic regression with hashed and crossed features, factorisation machines, or deep models with embeddings and explicit feature crosses (DCN, DeepFM). Train continuously or online on fresh data; negative downsampling with probability correction; evaluate with normalised log loss and calibration, then online revenue, CTR and advertiser ROI. Handle delayed clicks/conversions and click fraud. → *More:* Ch 18 §18.1 — Ad click prediction

Q205. ⋆ Design a real-time payment fraud-detection system. Score each transaction within ~100 ms before approval. Labels come from chargebacks and investigations, arriving weeks late, so mature labels lag and recent fraud patterns are learned slowly. Features: transaction amount, merchant, device and IP, velocity features from a streaming feature store ("number of cards used on this device in the last hour"), graph features linking accounts. Model: gradient-boosted trees, plus anomaly detection and hand rules for new attack patterns. Decisions: thresholds by expected cost, with approve / step-up authentication / review / block bands. Metrics: PR-AUC, recall at a fixed false-positive rate, dollars saved. Adversaries adapt, so monitor drift closely and retrain often. → *More:* Ch 18 §18.2 — Payment fraud detection

Q206. Design a harmful-content (content moderation) detection system. Multimodal classifiers (text, image, video, plus account signals) score each post at upload; high-confidence violations are removed automatically, uncertain ones go to human reviewers prioritised by predicted harm × reach, and appeals feed back as labels. Use multi-label outputs per policy, precision-oriented thresholds for automatic action, and recall measured by sampling. Handle adversarial obfuscation, many languages, label disagreement among reviewers and fairness across dialects. → *More:* Ch 18 §18.3 — Harmful content detection

Q207. Design "People You May Know" (friend recommendations). Treat it as link prediction on the social graph. Candidates from friends-of-friends, shared schools/workplaces and contact uploads; a ranker using graph features (mutual friends, common neighbours), profile similarity and interaction signals, or a graph neural network. Label: a sent and accepted request. Precompute candidates in batch for most users and refresh on new connections. Watch privacy (do not reveal sensitive relationships) and spam. → *More:* Ch 18 §18.4 — People You May Know

Q208. Design a visual search ("search by photo") system. Embed every catalogue image with a CNN or vision transformer fine-tuned with contrastive learning so the same or similar products land close together; store vectors in an ANN index; at query time detect and crop the object, embed it, retrieve nearest neighbours and re-rank by price, availability and metadata. Evaluate with recall@k on labelled pairs and online click-through and purchases. → *More:* Ch 18 §18.6 — Visual search

Q209. Design an ETA (estimated time of arrival) prediction system. Regression on route features (distance, road segments, historical and live speeds), time of day, weather and restaurant or pickup preparation time; a routing-engine estimate plus an ML correction is a strong design. Loss and metric: MAE or a quantile loss if being late hurts more than being early. Real-time features from streaming traffic data; evaluate per city and per time slot; monitor drift during holidays and events. → *More:* Ch 16 §16.11 — An ML platform for ETA prediction

Q210. What napkin math should you do in an ML design interview? Requests per second (daily active users × actions per day / 86,400, times 2–3 for peak), candidate counts per stage, latency budget per stage, feature-store reads per request, model size in memory, embedding index size (items × dimensions × 4 bytes for float32; 100 M × 128 × 4 ≈ 51 GB), and training data volume per day. It shows whether online scoring is feasible and where caching or batch precomputation is needed. → *More:* Ch 20 §20.13 — The 7-step framework and napkin math

Q211. How do you choose the right ML objective and metric for a product problem? Start with the business goal (revenue, retention, safety), find an observable proxy that the model can predict quickly (click, watch time, purchase), and add guardrail metrics to catch gaming (hides, complaints, latency). Check the proxy actually correlates with the goal; clicks alone reward clickbait. Combine several predicted outcomes in a tunable score so product owners can adjust weights without retraining. → *More:* Ch 15 §15.3, Ch 16 §16.7

Q212. What are the most common mistakes in ML design interviews? Jumping to a model before framing the problem and metric; ignoring how labels are obtained; forgetting latency and scale; not having a baseline; no plan for monitoring, retraining or cold start; ignoring position bias and feedback loops; and not discussing trade-offs. Interviewers reward explicit assumptions and choices you can justify. → *More:* Ch 15 §15.5 — Common mistakes

---

14. MLOps: deployment, monitoring, drift and A/B testing

Q213. ⋆ Batch vs online (real-time) vs streaming inference: how do you choose? Batch inference scores everything on a schedule and stores results (nightly recommendations, churn scores): cheap and simple, but stale and wasteful for users who never show up. Online inference scores on request through an API: fresh and personalised, but needs low-latency serving and features available in real time. Streaming inference scores events as they flow through a queue (fraud on each transaction). Many systems mix them: batch-precomputed candidates plus online ranking. → *More:* Ch 16 §16.5 — Batch vs online vs streaming inference

Q214. ⋆ What is training–serving skew and how do you prevent it? A difference between the data or feature logic the model saw in training and what it receives in production: features computed by different code paths, different time windows, missing values handled differently, or features available offline that are not available in time online. Prevent it by sharing one feature definition for both paths (a feature store), logging the exact features used at serving time and training on those logs, and checking online vs offline feature distributions. → *More:* Ch 16 §16.2 — Training vs serving

Q215. What is a feature store and why use one? A system that defines features once and serves them consistently: an offline store with full history for building training sets with point-in-time-correct joins (only feature values known before each label's timestamp), and an online low-latency store for serving the latest values. It removes training–serving skew, prevents leakage, and lets teams reuse features. → *More:* Ch 16 §16.4 — Features and feature stores

Q216. ⋆ What is data drift vs concept drift? How do you detect them? Data (covariate) drift: the distribution of inputs changes (new user demographics, a new device type). Concept drift: the relationship between inputs and the label changes (fraudsters change tactics, so the same features now mean something else). Label/prior drift: the class balance changes. Detect input drift with statistics per feature (population stability index, PSI; Kolmogorov–Smirnov test; null rates), prediction drift by monitoring the score distribution, and concept drift only through performance once labels arrive. Often the "drift" is actually a broken upstream pipeline. → *More:* Ch 16 §16.8 — Monitoring, drift and retraining

Q217. ⋆ How do you monitor a model in production? In layers: (1) system health: latency, errors, throughput, resource use; (2) data quality: schema checks, missing values, ranges, feature drift; (3) prediction distribution and its drift; (4) model performance on delayed labels, overall and per segment; (5) business metrics. Set alerts with sensible thresholds, keep dashboards per model version, and have a runbook: roll back, fall back to rules, or retrain. → *More:* Ch 16 §16.8

Q218. When and how do you retrain a model? On a schedule (daily, weekly), when monitoring shows a measured drop in performance or strong drift, or continuously for fast-changing domains like ads. Retrain through an automated pipeline: build a fresh dataset with point-in-time features, train, validate against the current model on a recent hold-out, then promote through shadow and canary stages. Keep versions so you can roll back. → *More:* Ch 16 §16.8

Q219. How do you safely deploy a new model? Shadow, canary, blue-green? Shadow deployment: the new model scores live traffic but its outputs are only logged, not used; compare with the current model and check latency. Canary: send a small share of real traffic (say 1–5%) to the new model and watch metrics before ramping up. Blue-green: run old and new environments side by side and switch traffic at once, with instant rollback. Then a proper A/B test measures the business impact. → *More:* Ch 16 §16.9–16.10

Q220. ⋆ How would you design an A/B test for a new model? Choose one primary metric plus guardrails; randomise by user (not by request) so each person has a consistent experience; compute sample size from the baseline variance and minimum detectable effect; run for whole weekly cycles; check the split ratio matches what you configured (sample-ratio mismatch signals a bug); analyse with a pre-registered test and confidence intervals; and watch for novelty effects. Ship if the primary metric improves and no guardrail degrades. → *More:* Ch 16 §16.9 — Online experiments and A/B testing

Q221. What problems can invalidate an A/B test? Peeking and stopping early when results look significant; testing many metrics without correction; sample-ratio mismatch; network effects or interference (in marketplaces and social networks, treated users affect control users; use cluster or switchback randomisation); novelty and learning effects; seasonality; and too little traffic for the effect size. Recommenders also suffer from shared training data between arms, since both models learn from the same logs. → *More:* Ch 16 §16.9

Q222. What is interleaving and when is it better than A/B testing? For ranking, interleaving mixes results from models A and B into one list for each user and counts which model's results get clicked. Because each user compares both, it needs far less traffic (often 10–100×) to detect which ranker is better. It tells you which is preferred, not the effect on business metrics, so it is used as a fast filter before a full A/B test. → *More:* Ch 16 §16.9, Ch 17 §17.3

Q223. What are multi-armed bandits used for in production, and how do they compare with A/B tests? Bandits shift traffic toward better-performing variants during the test, reducing the cost of showing a poor variant; good for short-lived choices (headlines, promotions) and for exploration in recommenders. A/B tests keep a fixed split and give cleaner, unbiased estimates of effect size, which matters for long-term product decisions. → *More:* Ch 14 §14.4, Ch 16 §16.9

Q224. What is MLOps? What should an ML pipeline include? MLOps applies DevOps practice to ML: versioned data, code and models; automated, reproducible training pipelines; experiment tracking (parameters, metrics, artefacts, for example MLflow); a model registry with stages; continuous integration tests for data and models; automated deployment with gates; and monitoring that can trigger retraining. The aim is that any model in production can be traced back and rebuilt. → *More:* Ch 16 §16.10 — MLOps

Q225. How do you serve a model with low latency? Keep the model small (distillation, quantisation), precompute what you can (item embeddings, batch features), cache frequent results, batch requests on GPUs, use an optimised runtime (ONNX Runtime, TensorRT, TorchServe/Triton), fetch features in parallel from an online store, set a per-stage latency budget, and define a fallback (cached or rule-based answer) when the model times out. → *More:* Ch 16 §16.6 — Model serving basics

Q226. Your model's performance dropped in production. How do you investigate? Confirm it is real (label delay? metric bug?). Check data pipelines and feature freshness first: schema changes, upstream outages, a feature suddenly null. Compare feature and prediction distributions to training. Slice performance by segment, country, device and time to localise it. Check for external causes (holiday, marketing campaign, new product, competitor) and for concept drift. Then fix the pipeline, roll back, or retrain on recent data. → *More:* Ch 16 §16.8

Q227. How do you handle delayed labels (for example fraud chargebacks arriving weeks later)? Monitor proxies in the meantime (prediction drift, early signals such as disputes or manual review results); train on matured labels while using a short window of recent data with partial labels carefully; use survival-style or importance-weighted corrections; and add fast-feedback components (rules, anomaly detectors) for new attack patterns. → *More:* Ch 16 §16.3, Ch 18 §18.2

Q228. How do you scale training to very large data or models? Data parallelism (copies of the model on many GPUs, each on a different mini-batch, with gradients averaged); model, tensor or pipeline parallelism when a model does not fit on one device; sharded optimisers (FSDP/ZeRO); mixed precision; gradient checkpointing; and efficient data loading. For classic ML, use distributed gradient boosting (XGBoost/LightGBM on Spark or Dask) or sample the data.

---

15. Responsible ML

Q229. Where does bias in ML systems come from? Historical bias (the world was unfair, and labels record it), representation and sampling bias (some groups under-represented), measurement and label bias (proxies like "arrests" for "crime"), aggregation bias (one model for groups that behave differently), evaluation bias (benchmarks that do not reflect users), and feedback loops after deployment. Removing the sensitive attribute does not help when other features act as proxies (ZIP code for race). → *More:* Ch 19 §19.1 — Where bias comes from

Q230. What fairness definitions do you know, and can they all be satisfied? Demographic parity: equal positive-prediction rates across groups. Equal opportunity: equal true positive rates. Equalised odds: equal true and false positive rates. Predictive parity / calibration within groups: equal precision or calibration. When base rates differ between groups, a non-perfect classifier cannot satisfy calibration and equalised odds simultaneously (the impossibility results), so you choose the criterion by the harm in your context and document why. → *More:* Ch 19 §19.2 — Fairness definitions and the impossibility trade-off

Q231. How do you mitigate bias? Before training (pre-processing): collect more representative data, reweigh or resample groups, fix label problems. During training (in-processing): add fairness constraints or penalties to the loss. After training (post-processing): group-specific thresholds or calibration. Always measure per-group metrics before and after, and check the accuracy cost. → *More:* Ch 19 §19.4 — Mitigating bias

Q232. ⋆ How do you explain a model's predictions? SHAP vs LIME vs permutation importance? Global: permutation importance (score drop when a feature is shuffled) and partial dependence plots (average prediction as one feature varies). Local: SHAP assigns each feature a contribution to a single prediction based on Shapley values from game theory, consistent and additive (contributions sum to the prediction minus the baseline); LIME fits a simple surrogate model around one prediction, faster but less stable. Explanations show what the model uses, not what causes the outcome. → *More:* Ch 19 §19.5 — Explainability

Q233. What is differential privacy and federated learning? Differential privacy adds calibrated noise (to data, gradients as in DP-SGD, or query results) so that the output barely changes whether or not any one person is in the data, with the privacy budget ε quantifying the guarantee. Federated learning trains on users' devices and sends only model updates to a server, so raw data never leaves the device; it is often combined with secure aggregation and differential privacy because updates can still leak information. → *More:* Ch 19 §19.6 — Privacy

Q234. What attacks on ML models do you know? Adversarial examples (tiny input changes that flip predictions, for example FGSM, which steps in the sign of the input gradient); data poisoning (injecting bad training data or backdoors); model extraction (copying a model through its API); model inversion and membership inference (recovering training data or learning whether a person was in it); and, for LLMs, prompt injection and jailbreaks. Defences include adversarial training, data provenance checks, rate limiting, output rounding and differential privacy. → *More:* Ch 19 §19.7 — Security: attacks on ML models

Q235. What is a model card? A short document published with a model that states its intended use and out-of-scope uses, training data, evaluation results including per-group metrics, known limitations,

ethical considerations and versioning. Datasheets do the same for datasets. They make limits visible to users and auditors and are increasingly expected by regulation such as the EU AI Act. → *More:* Ch 19 §19.8 — Model cards and datasheets, Ch 19 §19.9 — Regulation overview

---

16. Coding tasks commonly asked

These come up in "ML coding" rounds: implement from scratch with NumPy, usually in 20–30 minutes, then discuss complexity and edge cases. Snippets assume `import numpy as np`.

Q236. ⋆ Implement k-means from scratch.

```
def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)]          # random initial centroids
    for _ in range(iters):
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)  # n x k squared distances
        labels = d.argmin(1)
        newC = np.array([X[labels == j].mean(0) if (labels == j).any() else C[j] for j in range(k)])
        if np.allclose(newC, C): break
        C = newC
    return C, labels
```

Each iteration costs O(n·k·d). Mention k-means++ initialisation, multiple restarts, empty-cluster handling (kept here by reusing the old centroid) and feature scaling. → *More:* Ch 9 §9.1

Q237. ⋆ Implement logistic regression with gradient descent.

```
def sigmoid(z): return 1 / (1 + np.exp(-z))

def fit_logreg(X, y, lr=0.1, epochs=1000, l2=0.0):
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        w -= lr * (X.T @ (p - y) / len(y) + l2 * w)   # gradient of mean log loss + L2
        b -= lr * (p - y).mean()
    return w, b
```

The gradient (p − y)·x falls out of log loss with a sigmoid. Follow-ups: scale features, use `np.clip` or a stable sigmoid for large |z|, add mini-batches, and check convergence by tracking the loss. → *More:* Ch 5 §5.8

Q238. Implement linear regression (closed form and gradient descent).

```
def fit_ols(X, y):
    Xb = np.c_[np.ones(len(X)), X]                    # add intercept column
    return np.linalg.lstsq(Xb, y, rcond=None)[0]      # stable least squares

def fit_gd(X, y, lr=0.01, epochs=5000):
    Xb = np.c_[np.ones(len(X)), X]; th = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        th -= lr * 2 * Xb.T @ (Xb @ th - y) / len(y)  # gradient of MSE
    return th
```

Mention why `lstsq` beats explicitly inverting X^TX (numerical stability) and that gradient descent needs scaled features. → *More:* Ch 5 §5.3

Q239. ⋆ Compute precision, recall, F1 and accuracy from scratch.

```
def prf(y_true, y_pred):
    tp = np.sum((y_pred == 1) & (y_true == 1)); fp = np.sum((y_pred == 1) & (y_true == 0))
    fn = np.sum((y_pred == 0) & (y_true == 1)); tn = np.sum((y_pred == 0) & (y_true == 0))
    p = tp / (tp + fp) if tp + fp else 0.0
    r = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * p * r / (p + r) if p + r else 0.0
    return p, r, f1, (tp + tn) / len(y_true)
```

Handle the zero-division cases explicitly; interviewers check for them. → *More:* Ch 6 §6.3

Q240. ⋆ Compute ROC-AUC from scratch.

```
def roc_auc(y, s):
    order = np.argsort(s, kind="mergesort")
    ranks = np.empty(len(s)); ranks[order] = np.arange(1, len(s) + 1)
    for v in np.unique(s):                      # average ranks for tied scores
        m = s == v; ranks[m] = ranks[m].mean()
    n_pos, n_neg = y.sum(), len(y) - y.sum()
    return (ranks[y == 1].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg)
```

This uses the rank (Mann–Whitney) identity: AUC is the fraction of positive–negative pairs ranked correctly, computed in O(n log n) instead of O(n²) pair comparisons. → *More:* Ch 6 §6.3

Q241. Implement k-nearest neighbours classification.

```
def knn_predict(Xtr, ytr, Xte, k=5):
    d = ((Xte[:, None, :] - Xtr[None, :, :]) ** 2).sum(-1)   # m x n distances
    idx = np.argpartition(d, k - 1, axis=1)[:, :k]           # k nearest, unsorted
    return np.array([np.bincount(ytr[row]).argmax() for row in idx])  # labels 0..C-1
```

`argpartition` finds the k smallest in O(n) per query instead of a full sort (using `k - 1` as the pivot also works when k equals the training-set size). Mention memory (m × n matrix; process in chunks), scaling, ties, and KD-trees or ANN indexes for speed. → *More:* Ch 8 §8.1

Q242. Implement a numerically stable softmax and cross-entropy.

```
def softmax(z):
    z = z - z.max(axis=-1, keepdims=True)        # subtract max: avoids overflow in exp
    e = np.exp(z); return e / e.sum(axis=-1, keepdims=True)

def cross_entropy(logits, y):                    # y = integer class labels
    z = logits - logits.max(1, keepdims=True)
    logp = z - np.log(np.exp(z).sum(1, keepdims=True))   # log-softmax
    return -logp[np.arange(len(y)), y].mean()
```

Subtracting the maximum does not change the result but prevents `exp` overflow; computing log-softmax directly avoids log(0). → *More:* Ch 10 §10.3

Q243. Implement a decision-tree split search (best threshold by Gini) for one feature.

```
def gini(y): p = np.bincount(y) / len(y); return 1 - (p ** 2).sum()

def best_split(x, y):
    best = (None, np.inf)
    for t in np.unique(x)[:-1]:                  # candidate thresholds
        L, R = y[x <= t], y[x > t]
        score = (len(L) * gini(L) + len(R) * gini(R)) / len(y)
        if score < best[1]: best = (t, score)
    return best
```

This is O(u·n) for u unique values; real implementations sort once and update class counts incrementally for O(n log n). A full tree applies this to every feature and recurses. → *More:* Ch 7 §7.1

Q244. Implement a two-layer neural network forward and backward pass in NumPy.

```
def step(X, Y, W1, b1, W2, b2, lr=0.1):          # Y one-hot, n x classes
    H = np.maximum(0, X @ W1 + b1)               # ReLU hidden layer
    P = softmax(H @ W2 + b2)
    dZ2 = (P - Y) / len(X)                       # softmax + cross-entropy gradient
    dW2, db2 = H.T @ dZ2, dZ2.sum(0)
    dH = dZ2 @ W2.T; dH[H <= 0] = 0              # backprop through ReLU
    dW1, db1 = X.T @ dH, dH.sum(0)
    for p, g in ((W1, dW1), (b1, db1), (W2, dW2), (b2, db2)): p -= lr * g
```

Shows backpropagation as matrix calculus: each gradient is the upstream gradient times a local derivative. Check it with numerical gradient checking on a tiny network. → *More:* Ch 10 §10.4

Q245. Implement scaled dot-product attention.

```
def attention(Q, K, V, mask=None):              # Q: n x d, K: m x d, V: m x dv
    scores = Q @ K.T / np.sqrt(Q.shape[-1])
    if mask is not None: scores = np.where(mask, scores, -1e9)   # causal / padding mask
    return softmax(scores) @ V
```

Follow-ups: multi-head (reshape into h heads), causal mask (`np.tril`), and the O(n²) memory of the score matrix. → *More:* Ch 12 §12.6

Q246. Implement PCA with NumPy.

```
def pca(X, n_components):
    Xc = X - X.mean(0)                          # centre (standardise first if units differ)
    U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
    explained = S ** 2 / (S ** 2).sum()
    return Xc @ Vt[:n_components].T, explained[:n_components]
```

The rows of Vt are the principal directions; squared singular values are proportional to the variance each explains. SVD avoids forming the covariance matrix explicitly. → *More:* Ch 9 §9.5

Q247. Implement NDCG@k.

```
def ndcg_at_k(rels, k):                         # rels: relevance in ranked order
    rels = np.asarray(rels, float)
    disc = 1 / np.log2(np.arange(2, k + 2))
    dcg = (rels[:k] * disc[:len(rels[:k])]).sum()
    ideal = np.sort(rels)[::-1][:k]
    idcg = (ideal * disc[:len(ideal)]).sum()
    return dcg / idcg if idcg > 0 else 0.0
```

Some definitions use 2^rel − 1 as the gain; say which one you use. → *More:* Ch 6 §6.4

Q248. Implement stratified train/test split or k-fold indices without scikit-learn.

```
def stratified_kfold(y, k=5, seed=0):
    rng = np.random.default_rng(seed); folds = [[] for _ in range(k)]
    for c in np.unique(y):
        idx = rng.permutation(np.where(y == c)[0])
        for i, chunk in enumerate(np.array_split(idx, k)): folds[i].extend(chunk)
    all_idx = np.arange(len(y))
    return [(np.setdiff1d(all_idx, f), np.array(f)) for f in folds]
```

Each class is shuffled and dealt evenly across folds, so every fold keeps the class ratio. → *More:* Ch 6 §6.6

Q249. Implement mini-batch SGD for linear regression with early stopping.

```
def sgd(X, y, Xv, yv, lr=0.01, bs=32, epochs=100, patience=5, seed=0):
    rng = np.random.default_rng(seed); w = np.zeros(X.shape[1]); best, bad = (np.inf, w), 0
    for _ in range(epochs):
        for i in np.array_split(rng.permutation(len(X)), max(1, len(X) // bs)):
            w = w - lr * 2 * X[i].T @ (X[i] @ w - y[i]) / len(i)
        v = ((Xv @ w - yv) ** 2).mean()
        if v < best[0]: best, bad = (v, w.copy()), 0
        elif (bad := bad + 1) >= patience: break
    return best[1]
```

Shows shuffling each epoch, mini-batches, validation tracking and restoring the best weights. → *More:* Ch 5 §5.4, Ch 6 §6.10

Q250. Implement a Gaussian or multinomial naive Bayes classifier.

```python
def fit_mnb(X, y, alpha=1.0):                    # X: word-count matrix
    classes = np.unique(y)
    log_prior = np.log(np.array([(y == c).mean() for c in classes]))
    counts = np.array([X[y == c].sum(0) for c in classes]) + alpha    # Laplace smoothing
    log_lik = np.log(counts / counts.sum(1, keepdims=True))
    return classes, log_prior, log_lik

def predict_mnb(X, classes, log_prior, log_lik):
    return classes[(X @ log_lik.T + log_prior).argmax(1)]  # sum of logs, no underflow
```

Working in log space turns the product of many small probabilities into a sum, so it never underflows to zero; α > 0 keeps unseen words from zeroing out a class. → *More:* Ch 8 §8.3

Q251. How would you write a train/evaluate script for a new tabular problem in a live-coding round? Load data; split first (stratified or by time); build a `Pipeline` with a `ColumnTransformer` (impute + scale numerics, impute + one-hot categoricals); fit a baseline (logistic regression or `DummyClassifier`) and a gradient-boosted model (`HistGradientBoostingClassifier`); evaluate with cross-validation on the training set using the right metric; tune a few hyperparameters with randomised search; evaluate once on the test set; print a confusion matrix and permutation importances. Narrate each choice. → *More:* Ch 4 §4.10, Ch 1 §1.11 — A first end-to-end project

Q252. Which LeetCode-style problems are common in ML-engineer coding rounds? Besides ML-from-scratch tasks, MLE loops reuse standard data-structure problems: Top K Frequent Elements and K Closest Points to Origin (heaps, directly related to nearest-neighbour search), Merge Intervals, LRU Cache, Moving Average from Data Stream, Random Pick with Weight (weighted sampling via prefix sums and binary search), Kth Largest Element (quickselect), sparse-vector dot product, and matrix or graph traversal (BFS/DFS, connected components). Expect also SQL window-function questions for data roles.

Sources of the question lists

Question wording only (no answers) was gathered from these public lists; every answer above is written for this book.

- alexeygrigorev/data-science-interviews — "Theoretical interview questions" (GitHub, `theory.md`)
- andrewekhalel/MLQuestions — "Machine Learning Interview Questions and Answers" (GitHub)
- khangich/machine-learning-interview — "Minimum Viable Study Plan for Machine Learning Interviews" (GitHub; probability questions and the ML system design prompts: video recommendation, feed ranking, ad click prediction, delivery-time estimation, search ranking, fraud detection)
- LeetCode problem titles listed in Q252 (Top K Frequent Elements, K Closest Points to Origin, Random Pick with Weight, LRU Cache, Merge Intervals, Moving Average from Data Stream)